UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

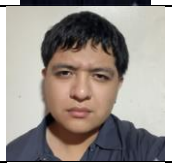
SISTEMAS OPERATIVOS

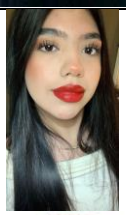
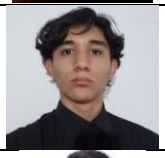
Actividad Fundamental 2: Multitarea y control de concurrencia

DOCENTE: DRA. NORMA EDITH MARIN MARTINEZ

EQUIPO N°3

Link Video: https://youtu.be/ilfw_ptiiCA

Integrantes	Foto	Matricula	% de participación
Camacho Moreno Jesús Emmanuel		2153842	100%
Cárdenas Vázquez Carlos Tomas		2226997	100%
Cerda Herrera Juan Kristof		2050864	100%
Espindola Martínez Juan		2153695	100%
Garza Arizpe Emilio		2143702	100%
Garza Fernández Vanessa Judith		2145305	100%

González Martínez Mario Iván		2117170	100%
González Parra Sylvia Yadira		2142962	100%
López Guzmán Eder Abihú		2116464	100%
Peña Mendoza Cesar Maximiliano		2144135	100%
Pérez Ordaz Andrés Daniel		2163200	100%
Salazar Torres Isaias Leonardo		2227016	100%

Fecha: 25/08/2026

Índice

Introducción 6

Forma de trabajar los procesos en el sistema operativo 7

- Procesos e Hilos 7
- Estados 9
- Transiciones 10
- Control y Gestión 12
- Multiprocesamiento Simétrico 21
- Micronúcleos 27
- Windows 30
  - Gestión de Hilos en Windows 30
  - SMP en Windows 31
- Linux 32
  - Gestión de Hilos en Linux 32
  - SMP en Linux 34
- UNIX/Solaris 35
  - Gestión de Hilos en UNIX/Solaris 35
  - SMP en UNIX/Solaris 36

Investigación de Concurrencia 37

- Definición de concurrencia 37
- Tipos de concurrencia 38
- Modelos de programación concurrente 40
- Ventajas de la ejecución de concurrencia 54
- Tipos de procesos concurrentes 56

Tipos de interacciones entre procesos 59
Cuadro Sinóptico 60
Conclusión General 61
Conclusiones Individuales 62
Bibliografía 74

Índice de Imágenes

Imagen 1. Diagrama de estados y transiciones de un proceso. 12
Imagen 2. Bloque de Control del Proceso. 15
Imagen 3. Modo Usuario vs Modo Núcleo..... 16
Imagen 4. Arquitectura de un Sistema de Multiprocesamiento Simétrico... 22
Imagen 5. Clasificación de Flynn para arquitecturas de computadores..... 23
Imagen 6. Taxonomía y clasificación de los procesadores paralelos. 24
Imagen 7. Arquitectura de multiprocesamiento maestro/esclavo (Master/Slave). 25
Imagen 8. Organización de un sistema multiprocesador de bus único. 25
Imagen 9. Estructura de un sistema multiprocesador simétrico con cachés de dos niveles (L1 y L2) y subsistema de E/S. 26
Imagen 10. Comparación entre las arquitecturas de núcleo por capas (monolítico) y micro núcleo (microkernel). 28
Imagen 11. Mecanismo de gestión de fallos de página mediante IPC en una arquitectura basada en micronúcleo. 29
Imagen 12. Sistema Operativo Linux. 32
Imagen 13. Diferentes niveles de concurrencia: multiprogramación, multiproceso y procesamiento distribuido..... 37
Imagen 14. Cuadro Sinóptico acerca de la concurrencia en Sistemas Operativos..... 60

Introducción

En la actualidad, prácticamente cualquier sistema operativo tiene que lidiar con la ejecución de varias tareas al mismo tiempo, ya sea en una sola computadora con un solo procesador o en equipos con varios núcleos trabajando en conjunto. Este comportamiento no es casualidad ni algo que simplemente "ya viene incluido" en el software: es el resultado de años de desarrollo enfocados en resolver un problema muy concreto, que es cómo lograr que múltiples procesos avancen sin que unos interfieran con otros, sin desperdiciar recursos del sistema y sin que el usuario perciba fallas o lentitud.

En este documento se abordará, en primer lugar, la forma en que el sistema operativo gestiona sus procesos: cómo los crea, los controla y cambia entre ellos, así como las diferentes arquitecturas que existen para organizar el núcleo del sistema, incluyendo su implementación en Windows, Linux y UNIX/Solaris. Posteriormente, se profundizará en el tema de la concurrencia, cubriendo su definición, los tipos que existen, los distintos modelos de programación concurrente que se han desarrollado a lo largo de los años, las ventajas que ofrece este tipo de ejecución, los tipos de procesos concurrentes y la forma en que estos interactúan entre sí. Con esto se busca ofrecer un panorama completo de uno de los pilares más importantes en el diseño de cualquier sistema operativo moderno.

Forma de trabajar los procesos en el sistema operativo

Procesos e Hilos

Procesos

Un proceso es un programa que ya se está ejecutando. No es solo el código en sí, sino todo lo que ese código necesita para funcionar: información sobre su estado, recursos del sistema, etc. Cuando un programa está guardado en el disco y nadie lo ha abierto se le llama entidad pasiva, osea que no hace nada por sí solo. Y cuando el sistema operativo lo carga a memoria y comienza a correrlo, ahí sí se convierte en un proceso o en algo activo que el sistema puede planificar y controlar. Para funcione todo esto cada proceso necesita algunos recursos: procesador, memoria, archivos, dispositivos de entrada/salida, etc. Es tarea del sistema operativo dárselos y administrarlos mientras el proceso se ejecuta. Por eso no conviene pensar en un proceso solamente como "el código del programa". Es más bien la combinación de ese código junto con toda la información que se necesita para controlarlo mientras corre.

Hilos

Un hilo (o thread) es una unidad de ejecución que vive dentro de un proceso. Un proceso puede tener nada más un hilo, o puede tener varios, lo que permite que dentro de una misma aplicación se estén realizando distintas actividades al mismo tiempo. Los hilos que pertenecen al mismo proceso comparten varios recursos, sobre todo el espacio de direcciones y otros recursos asociados al proceso. Cada hilo guarda su propia información para poder controlar su ejecución: su contador de programa, sus registros del procesador y su propia pila o stack. Gracias a esta separación, varios hilos de un mismo proceso pueden llevar flujos de ejecución distintos aunque estén usando los mismos recursos. Por ejemplo, una aplicación puede tener un hilo procesando información mientras otro se encarga de una

operación de entrada/salida. Así, usar hilos permite dividir una aplicación en varias actividades sin necesidad de crear un proceso aparte para cada una.

Relación entre procesos e hilos

Aunque están relacionados, procesos e hilos no son lo mismo. El proceso pone el entorno y los recursos necesarios; los hilos son los flujos de ejecución que ocurren dentro de ese entorno. Un proceso puede tener uno o varios hilos. Si tiene solo uno la ejecución es de un único flujo. Y si tiene varios esos hilos pueden avanzar de forma concurrente, compartiendo los recursos del proceso al que pertenecen. Ya que los hilos de un mismo proceso comparten información y recursos con más facilidad que si fueran procesos separados, también obliga a que tanto el sistema operativo como las aplicaciones manejen bien el acceso a esos recursos compartidos porque ahí es donde pueden surgir problemas.

Multihilo

El multihilo es justamente permitir que un solo proceso tenga varios hilos de ejecución corriendo. Cada hilo individualmente se puede encargar de una actividad distinta, pero todos siguen perteneciendo al mismo proceso y compartiendo ciertos recursos. Esto ayuda a organizar aplicaciones que hacen varias cosas a la vez y a aprovechar mejor los recursos que se tienen disponibles. También ayuda a que la aplicación responda mejor. Por ejemplo, si una actividad tiene que esperar a que termine una operación de entrada/salida, esa espera la puede manejar un hilo mientras otro sigue trabajando en algo distinto. En sistemas que tienen varios procesadores o núcleos, los hilos incluso pueden llegar a ejecutarse en paralelo de verdad. Cómo funciona esto exactamente y su relación con el multiprocesamiento se ve más adelante, en el apartado correspondiente.

Procesos e hilos como unidades de ejecución

Visto desde el sistema operativo, procesos e hilos cumplen funciones distintas dentro de cómo se administra la ejecución. El proceso arma el entorno de ejecución con sus recursos, mientras que el hilo es el flujo de instrucciones que

realmente ejecuta el procesador. Esta forma de organizar las cosas separa la administración de recursos de la ejecución de actividades en sí, entonces permite que una aplicación tenga distintos flujos de ejecución dentro de un mismo entorno. Por eso los hilos son tan importantes en los sistemas operativos actuales: las aplicaciones necesitan hacer varias cosas al mismo tiempo y aprovechar al máximo las capacidades de los procesadores de hoy en día.

Estados

- **Nuevo:** Es el estado en el que se encuentra un proceso que acaba de ser creado. El sistema operativo ya realizó las tareas necesarias para crear y administrar el proceso, pero todavía no ha sido admitido para ejecutarse.
- **Listo:** El proceso está preparado para ejecutarse, pero está esperando que el sistema operativo le asigne el procesador. El planificador selecciona uno de los procesos que se encuentran en este estado.
- **Ejecutando:** El proceso está utilizando actualmente el procesador. Puede permanecer en este estado hasta que termine su tiempo de ejecución, sea expulsado o necesite esperar algún evento.
- **Bloqueado:** El proceso no puede continuar ejecutándose porque está esperando que ocurra un evento determinado, como la finalización de una operación de entrada o salida. Cuando ocurre el evento, el proceso vuelve al estado Listo.
- **Saliente:** Es el estado al que pasa un proceso cuando termina su ejecución, es abortado o se produce alguna condición que provoca su terminación. Después de esto, ya no puede volver a ejecutarse.
- **Listo/Suspendido:** Es un proceso que ha sido retirado de la memoria principal, pero que está preparado para ejecutarse cuando vuelva a ser cargado en memoria.

- Bloqueado/Suspendido: Es un proceso que ha sido retirado de la memoria principal mientras esperaba algún evento. Cuando ocurre ese evento, puede pasar al estado Listo/Suspendido.

En conjunto, los procesos pueden pasar de un estado a otro dependiendo de acciones del sistema operativo, de la disponibilidad del procesador, de operaciones de entrada/salida y de los eventos que estén esperando.

Transiciones

Mientras exista un proceso, este va pasando por distintos estados (nuevo, listo, ejecutando, bloqueado, terminado...). Una transición es el cambio de un estado a otro, provocado siempre por algún evento: que el planificador lo elija, que se le acabe su turno de CPU, que pida una operación de E/S, que esa E/S termine, etc.

Cuando ocurre una transición, el sistema operativo realiza una serie de acciones internas y operativas en el hardware y en el software para adaptar el contexto del procesador y los recursos asignados.

El modelo fundamental contempla cinco estados principales

Estado	Descripción
Nuevo	El proceso se está creando (se le asigna BCP, pero aún no está listo para ejecutar).
Listo (Ready)	El proceso está en memoria principal y listo para ejecutarse cuando el planificador lo seleccione.
Ejecutando	El proceso está usando la CPU en este momento.
Bloqueado / Esperando	El proceso no puede continuar porque espera un evento (E/S, señal, recurso, etc.).
Terminado / Saliente	El proceso ha finalizado su ejecución o ha sido abortado.

Tabla 1. Estados del ciclo de vida de un proceso y sus descripciones.

¿Qué eventos causan una transición?

Existen 4 tipos de causa:

- El proceso pide algo que no puede completarse de inmediato → se bloquea a sí mismo (Ejecutando → Bloqueado).
- Se acaba su tiempo de CPU o llega uno de mayor prioridad → el planificador lo saca de ejecución (Ejecutando → Listo). A esto se le llama expulsión.
- El planificador elige un proceso listo para correr → Listo → Ejecutando.
- Ocurre el evento que un proceso bloqueado estaba esperando (termina la E/S, llega el dato, se libera el recurso) → Bloqueado → Listo.

Cuando ocurre la interrupción que dispara la transición, el SO tiene que:

1. Guardar todo el estado del procesador del proceso saliente (contador de programa, registros, etc.) en su PCB, si no se guarda, ese proceso jamás podría retomarse exactamente dónde iba.
2. Ejecutar la rutina de tratamiento de esa interrupción.
3. Si corresponde, actualizar el campo de estado del proceso y moverlo a la cola que le toca (Listos, Bloqueados, etc.).
4. Si el SO decide poner a correr otro proceso, restaura el contexto guardado de ese otro proceso.

Transiciones y su causa:

- **Nuevo → Listo:** el SO admite al proceso (ya tiene su PCB, hay memoria/cupo disponible).
- **Listo → Ejecutando:** el planificador lo selecciona.
- **Ejecutando → Listo:** se acaba su porción de tiempo, o llega un proceso de mayor prioridad y lo expulsa ("apropiación"/*preemption*).

- **Ejecutando → Bloqueado:** el proceso solicita algo que debe esperar (E/S, un recurso, un mensaje de otro proceso).
- **Bloqueado → Listo:** ocurre el evento que esperaba (termina la E/S, llega el mensaje).
- **Ejecutando → Saliente:** el proceso termina normalmente o es abortado.

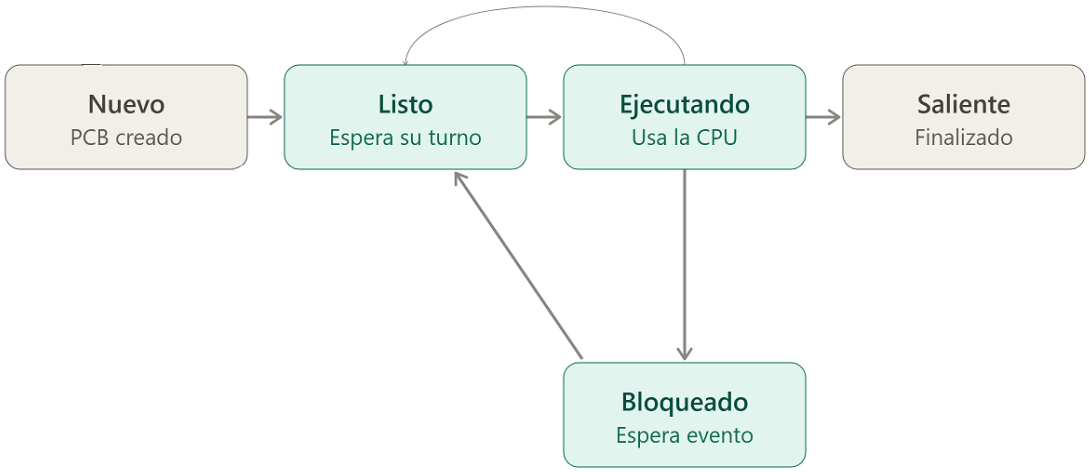


Imagen 1. Diagrama de estados y transiciones de un proceso.

Control y Gestión

Estructuras de control de proceso

Para poder gestionar los procesos y los recursos del sistema, el sistema operativo necesita disponer de información actualizada sobre el estado de cada uno de ellos. Para lograrlo, construye y mantiene un conjunto de tablas que registran esta información. De manera general, se distinguen cuatro tipos de tablas: de memoria, de entrada/salida, de ficheros y de procesos. Aunque cada sistema operativo las implementa de forma distinta, todos manejan estas cuatro categorías básicas de información, y dichas tablas están interrelacionadas entre sí, ya que la memoria, los dispositivos de E/S y los ficheros son recursos que finalmente están siendo utilizados por los procesos.

Las tablas de memoria registran las reservas de memoria principal y secundaria asignadas a cada proceso, así como los atributos de protección necesarios para permitir el acceso controlado a zonas compartidas. Las tablas de E/S permiten conocer el estado de los dispositivos y canales del sistema, indicando si se encuentran disponibles o asignados a algún proceso. Las tablas de ficheros aportan información sobre la existencia, ubicación y estado de los archivos del sistema. Finalmente, las tablas de procesos (las más relevantes para el control y la gestión) permiten al sistema operativo administrar cada proceso en ejecución.

Para poder controlar un proceso, el sistema operativo necesita saber dos cosas: dónde se encuentra localizado y cuáles son sus atributos. La representación física de un proceso está compuesta por el programa a ejecutar, los datos asociados (variables locales y globales), una pila utilizada para gestionar llamadas a procedimientos, y un conjunto de atributos de control conocido como **Bloque de Control de Proceso (BCP)**. Al conjunto formado por el programa, los datos, la pila y el BCP se le denomina **imagen del proceso**.

La ubicación de esta imagen depende del esquema de gestión de memoria empleado por el sistema. En los casos más simples se mantiene como un bloque contiguo en memoria secundaria, mientras que en los sistemas que utilizan paginación, distintas partes de la imagen del proceso pueden residir simultáneamente en memoria principal y secundaria. Para organizar esta información, el sistema operativo mantiene una tabla primaria de procesos, en la que cada entrada contiene, como mínimo, un puntero hacia la imagen del proceso correspondiente.

El Bloque de Control de Proceso (BCP)

El BCP constituye la estructura de datos más importante dentro del sistema operativo, ya que reúne toda la información necesaria para que este pueda controlar un proceso. Prácticamente todos los módulos del sistema operativo planificación, gestión de recursos, procesamiento de interrupciones, monitorización del

rendimiento, entre otros acceden o modifican esta estructura en algún momento. La información contenida en el BCP puede agruparse en tres grandes categorías:

- **Identificación del proceso:** Incluye identificadores numéricos como el identificador único del proceso, el identificador del proceso padre (que lo creó) y el identificador del usuario responsable de dicho proceso. Estos identificadores permiten que otras tablas del sistema (memoria, E/S, ficheros) puedan hacer referencia a procesos concretos.

- **Información de estado del procesador:** Comprende los registros visibles por el usuario, los registros de estado y control (como el contador de programa, que indica la dirección de la siguiente instrucción a ejecutar, y los códigos de condición resultantes de operaciones aritméticas o lógicas) y el puntero de pila, que señala la posición más alta de la pila del proceso. Todos los procesadores incluyen además una palabra de estado de programa (PSW) que concentra esta información de estado; en los procesadores Pentium, por ejemplo, esta función la cumple el registro EFLAGS.

- **Información de control del proceso**: Es la categoría más amplia e incluye datos como el estado actual del proceso (Ejecutando, Listo, Esperando, Detenido), su prioridad de planificación, información relacionada con los algoritmos de planificación utilizados, punteros que permiten enlazar el BCP con otros procesos dentro de colas o estructuras similares, información necesaria para la comunicación entre procesos, los privilegios asignados, punteros relacionados con la gestión de memoria y un registro de los recursos utilizados o asignados al proceso.

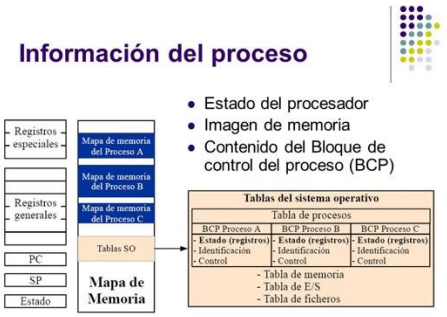


Imagen 2. Bloque de Control del Proceso.

Dado que el BCP es constantemente accedido y modificado por múltiples componentes del sistema operativo, su protección resulta crítica: un error en alguna rutina por ejemplo, en un manejador de interrupciones podría corromper esta estructura y comprometer la capacidad del sistema para administrar los procesos afectados. Por ello, muchos sistemas operativos canalizan las operaciones de lectura y escritura sobre los BCP a través de una única rutina encargada de arbitrar dicho acceso.

Modos de ejecución.

Para comprender cómo el sistema operativo controla los procesos, es necesario distinguir entre los distintos modos de ejecución del procesador. La mayoría de los procesadores modernos ofrecen, como mínimo, dos modos: el **modo usuario**, en el que se ejecutan habitualmente los programas de los usuarios y que presenta restricciones de acceso, y el **modo núcleo** (también llamado modo sistema o modo supervisor), reservado para las funciones más críticas del sistema operativo. En este segundo modo, el software tiene control total sobre el procesador, sus registros, instrucciones y la memoria.



Imagen 3. Modo Usuario vs Modo Núcleo.

Esta separación existe para proteger al sistema operativo y a sus estructuras de datos clave como los BCP de posibles interferencias provocadas por los programas de usuario. El modo de ejecución en el que se encuentra el procesador queda registrado mediante un bit específico dentro de la palabra de estado de programa (PSW). Este bit cambia automáticamente cuando ocurre una llamada al sistema operativo o cuando se produce una interrupción que activa una rutina del sistema; una vez finalizado el servicio correspondiente, el modo regresa nuevamente a modo usuario.

Como ejemplo concreto, en los procesadores Itanium (arquitectura IA-64) existe un registro de estado que incluye un campo de dos bits llamado *cpl* (nivel de privilegio actual), donde el nivel 0 corresponde al mayor privilegio y el nivel 3 al menor. Sistemas operativos como Linux emplean el nivel 0 para el núcleo y alguno de los niveles restantes para los procesos de usuario.

Finalmente, el núcleo del sistema operativo concentra un conjunto de funciones fundamentales, entre las que destacan: la creación y terminación de procesos, la planificación y activación de procesos, el intercambio de procesos (swapping), la sincronización y comunicación entre procesos, y la gestión de los propios bloques de control de proceso, además de funciones relacionadas con la gestión de memoria y de entrada/salida.

Creación de procesos.

Una vez que el sistema operativo determina que debe crearse un nuevo proceso, procede a través de una secuencia de pasos concretos. En primer lugar, asigna al proceso un identificador único, momento en el cual se añade una nueva entrada a la tabla primaria de procesos. A continuación, reserva el espacio necesario para la imagen completa del proceso, lo cual implica determinar la cantidad de memoria requerida tanto para el espacio de direcciones privado —donde residen el programa y los datos— como para la pila de usuario. Estos valores pueden fijarse por defecto según el tipo de proceso o bien especificarse explícitamente en la solicitud de creación; cuando un proceso es generado por otro proceso, el proceso padre puede transmitir los parámetros necesarios al sistema operativo como parte de dicha solicitud. En este mismo paso se reserva también el espacio correspondiente al Bloque de Control de Proceso.

El tercer paso consiste en la inicialización del BCP. La parte de identificación se completa con el identificador del proceso y el de su proceso padre. La información de estado del procesador se inicializa, en su mayoría, con valores en cero, exceptuando el contador de programa (que se fija en el punto de entrada del programa) y los punteros de pila, que quedan establecidos para delimitar los márgenes de la pila del proceso. La información de control del proceso se inicializa con base en valores por omisión: el estado del proceso normalmente se fija como Listo o Listo/Suspendido, y la prioridad se asigna, por defecto, al nivel más bajo, salvo que se solicite explícitamente un nivel superior. En un inicio, el proceso no dispone de ningún recurso asignado, a menos que así se indique de forma expresa o que dichos recursos hayan sido heredados de su proceso padre.

El cuarto paso consiste en establecer los enlaces correspondientes; por ejemplo, si el sistema operativo organiza las colas del planificador como listas enlazadas, el nuevo proceso debe incorporarse a la cola de procesos Listos o Listos/Suspendidos. Finalmente, se crean o amplían otras estructuras de datos que

puedan resultar necesarias, como los registros de auditoría empleados posteriormente para tareas de facturación o análisis del rendimiento del sistema.

La creación de un proceso puede originarse por distintas circunstancias: la llegada de un nuevo trabajo por lotes, el inicio de una sesión interactiva por parte de un usuario, la necesidad del propio sistema operativo de generar un proceso para prestar un servicio determinado, o la solicitud de un proceso ya existente que decide generar uno nuevo, estableciéndose entre ambos una relación de proceso padre y proceso hijo.

Cambio de proceso

Aunque a primera vista el cambio de proceso pudiera parecer una operación sencilla, su análisis exige responder a tres preguntas fundamentales: qué evento origina el cambio, cuál es la diferencia entre cambio de proceso y cambio de modo, y qué acciones debe realizar el sistema operativo sobre las estructuras que gestiona para llevarlo a cabo.

¿Cuándo ocurre un cambio de proceso?

Un cambio de proceso puede producirse en cualquier instante en que el sistema operativo recupera el control sobre el proceso en ejecución. Esto ocurre principalmente a través de tres mecanismos: las interrupciones, los traps y las llamadas al sistema. Las interrupciones se originan por eventos externos e independientes al proceso en ejecución, como la finalización de una operación de entrada/salida o el vencimiento de la rodaja de tiempo asignada a un proceso. Los traps, en cambio, están asociados a una condición de error o excepción generada por el propio proceso en ejecución, como un intento de acceso indebido a un archivo; si dicha condición resulta irreversible, el proceso pasa al estado Saliente y se produce un cambio de proceso. Por último, las llamadas al sistema son solicitudes explícitas realizadas por el proceso en ejecución para invocar una función del sistema operativo, las cuales pueden en algunos casos derivar en que el proceso pase al estado Bloqueado.

Diferencia entre cambio de modo y cambio de proceso

Cuando ocurre una interrupción, el procesador transfiere el control a la rutina encargada de atenderla, cambiando del modo usuario al modo núcleo para poder ejecutar instrucciones privilegiadas. En este punto se salvaguarda el contexto del proceso interrumpido —es decir, la información de estado del procesador que podría verse alterada durante la ejecución de la rutina de interrupción— dentro de su respectivo Bloque de Control de Proceso. Es importante señalar que la ocurrencia de una interrupción no implica, necesariamente, que se produzca un cambio de proceso: en muchos casos, una vez atendida la interrupción, la ejecución se reanuda con el mismo proceso, únicamente incurriendo en la sobrecarga de guardar y restaurar el estado del procesador.

Pasos del cambio de proceso completo

Cuando el proceso que se encuentra en ejecución debe transitar hacia otro estado (Listo, Bloqueado, entre otros), el sistema operativo realiza una serie de acciones más sustanciales:

1. Guardar el estado del procesador, incluyendo el contador de programa y los demás registros.
2. Actualizar el Bloque de Control de Proceso correspondiente al proceso que estaba en ejecución, modificando su estado hacia el que corresponda (Listo, Bloqueado, Listo/Suspendido o Saliente) y registrando la razón de dicho cambio.
3. Trasladar el Bloque de Control de Proceso a la cola apropiada según su nuevo estado.
4. Seleccionar un nuevo proceso para su ejecución, tarea que corresponde al planificador del sistema.
5. Actualizar el Bloque de Control de Proceso del proceso seleccionado, cambiando su estado a Ejecutando.
6. Actualizar, de ser necesario, las estructuras de datos relacionadas con la gestión de memoria.

7. Restaurar el estado del procesador correspondiente al proceso seleccionado, recuperando los valores previos del contador de programa y de los registros, de modo que la ejecución continúe exactamente en el punto donde había quedado.

Por consiguiente, un cambio de proceso —que implica una transición real de estado— exige un esfuerzo considerablemente mayor que un simple cambio de modo.

Ejecución del sistema operativo

Dado que el sistema operativo está compuesto, en esencia, por un conjunto de programas ejecutados por el propio procesador, surge la interrogante de si el sistema operativo constituye en sí mismo un proceso y, de ser así, de qué manera se controla. Frente a esta cuestión, existen tres enfoques de diseño distintos, adoptados por diferentes sistemas operativos.

Núcleo sin procesos

Es la concepción tradicional presente en numerosos sistemas operativos antiguos, en la cual el sistema operativo se ejecuta completamente fuera de cualquier proceso de usuario. Cuando un proceso en ejecución es interrumpido o invoca una llamada al sistema, su contexto se guarda y el control se transfiere al núcleo, el cual dispone de su propia región de memoria y de su propia pila del sistema. Una vez que el núcleo concluye sus funciones, puede restaurar el contexto del proceso interrumpido o bien activar un proceso diferente. Bajo esta concepción, el concepto de proceso se aplica exclusivamente a los programas de usuario.

Ejecución dentro de los procesos de usuario

Este enfoque, común en sistemas operativos de equipos personales y estaciones de trabajo, consiste en ejecutar prácticamente todo el software del sistema operativo dentro del contexto de un proceso de usuario, empleando para ello una pila del núcleo independiente para el manejo de llamadas y retornos en modo núcleo. Cuando ocurre una interrupción, trap o llamada al sistema, el procesador cambia a modo núcleo, pero la ejecución continúa dentro del proceso de usuario actual, sin que ello implique necesariamente un cambio de proceso, sino un cambio de modo dentro del mismo proceso. Solo si el sistema operativo determina que corresponde ejecutar un proceso distinto se transfiere el control a la rutina de cambio de proceso correspondiente.

Sistemas operativos basados en procesos

Bajo este tercer enfoque, el sistema operativo se implementa como un conjunto de procesos del sistema independientes, cada uno de los cuales se ejecuta en modo núcleo, requiriéndose además una pequeña porción de código para el intercambio entre procesos que se ejecuta fuera de todos ellos. Este modelo favorece un diseño modular con interfaces claras entre los distintos componentes del sistema operativo, y resulta particularmente ventajoso en entornos de multiprocesadores y multicomputadores, donde ciertos servicios del sistema pueden asignarse a procesadores dedicados para mejorar el rendimiento general.

Multiprocesamiento Simétrico

El multiprocesamiento simétrico es un método de organizar un sistema multiprocesador de tal forma que cualquier proceso (o hilo) puede ejecutar en cualquier procesador; esto incluye al código del núcleo y a los procesos.

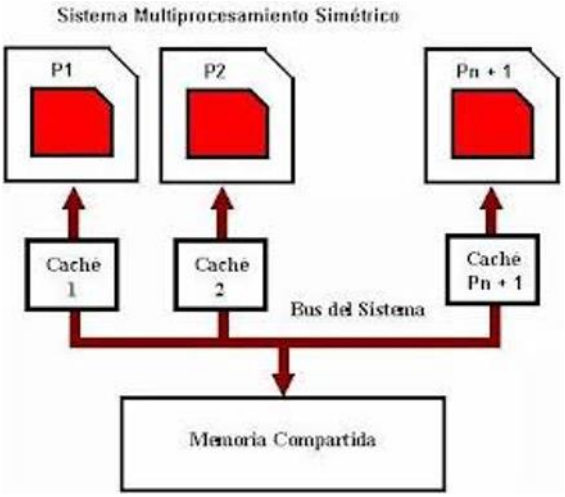


Imagen 4. Arquitectura de un Sistema de Multiprocesamiento Simétrico.

ARQUITECTURA SMP

Para poder saber cuál es el tipo de arquitectura que utiliza el SMP, tenemos que conocer primero la taxonomía de sistemas de procesamiento paralelo de Flynn que el propone las siguientes categorías de sistemas de computadores:

SISD:*(Single Instruction Single Data)* Esta clasificación se refiere a las computadoras tradicionales y secuenciales en las cuales una instrucción a la vez se ejecuta sobre un único dato cada ciclo de reloj. Los datos en cuestión se almacenan en una única memoria en la cual se usan técnicas como la segmentación para evitar errores de fragmentación interna. Un ejemplo sencillo de estas computadoras son los antiguos mainframe basados en la arquitectura de Von-Neumann.

MISD:*(Multiple Instruction Single Data)* Arquitectura que se refiere a múltiples instrucciones ejecutándose sobre un único dato. Comúnmente se considera esta arquitectura poco práctica ya que en tiempo de ejecución la efectividad del paralelismo requiere un múltiple flujo de datos y, además, el acceso concurrente a un mismo dato en memoria puede ocasionar que un CPU tenga que esperar a que el recurso(dato) esté disponible para poder acceder a él.

SIMD:*(Single Instruction Multiple Data)* Esta arquitectura representa la ejecución de una misma instrucción sobre un conjunto de datos. La misma es comúnmente vista en ciclos de programación que ejecutan una misma instrucción una y otra vez sobre datos de un arreglo o conjunto de datos. En la arquitectura SIMD estos datos son procesados por múltiples CPU que ejecutan la misma instrucción sobre una parte del conjunto o arreglo, cada uno, hasta llegar a procesar la totalidad de los mismos.

MIMD:*(Multiple Instruction Multiple Data)* Esta arquitectura representa a un conjunto de instrucciones que se ejecutan sobre un conjunto múltiple de datos. La misma es muy usada hoy en día para explotar el paralelismo ya sea con memoria distribuida y memoria compartida o híbridos como los clústers de computadoras. Muchos multiprocesadores modernos (como los de la tecnología Core i de Intel) entran en esta clasificación.

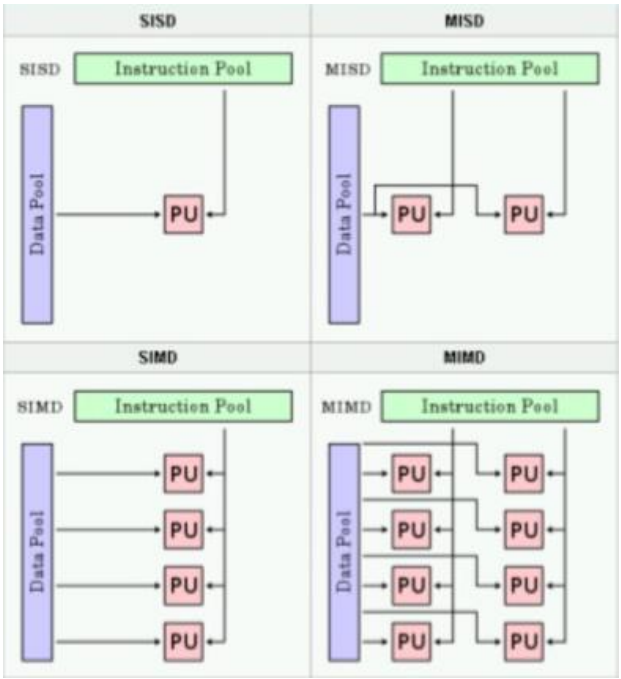


Imagen 5. Clasificación de Flynn para arquitecturas de computadores.

El SMP pertenece a la categoría MIMD. Dentro de la arquitectura MIMD, los sistemas se dividen por cómo se comunican sus procesadores. Si la memoria está distribuida, hablamos de *Clusters*. Si la memoria es compartida (sistemas fuertemente acoplados), los procesadores acceden a un espacio de memoria común.

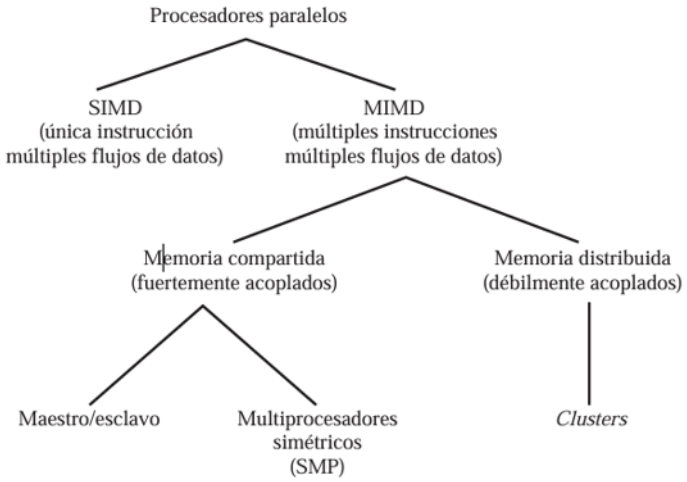


Imagen 6. Taxonomía y clasificación de los procesadores paralelos.

En los multiprocesadores de memoria compartida existen dos enfoques principales para asignar los procesos:

- **Arquitectura Maestro/Esclavo:** El núcleo del sistema operativo corre únicamente en un procesador 'maestro'. Los demás son 'esclavos' que solo ejecutan programas de usuario. Si un esclavo requiere E/S o un servicio del sistema, hace una petición al maestro. Las desventajas son evidentes: si el maestro falla, todo el sistema cae, y el maestro se convierte en un cuello de botella de rendimiento.

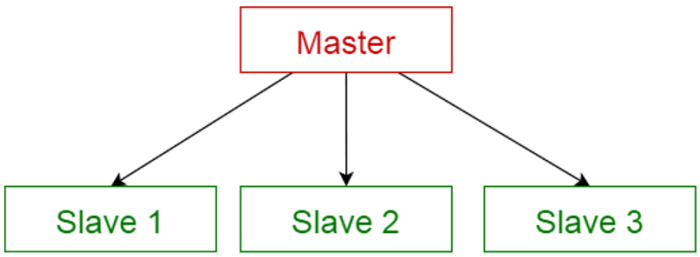


Imagen 7. Arquitectura de multiprocesamiento maestro/esclavo (Master/Slave).

- **Multiprocesamiento Simétrico (SMP):** A diferencia del modelo maestro/esclavo, en SMP el núcleo del sistema operativo puede ejecutarse en **cualquier procesador**. Todos los procesadores poseen capacidades funcionales completas y comparten la misma cola de trabajos. Esto permite un balanceo de carga automático y elimina el punto único de falla.

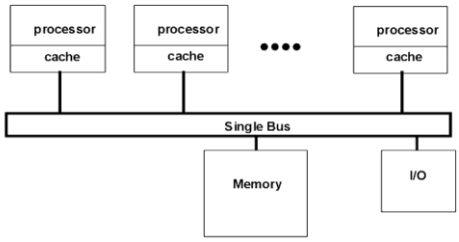


Imagen 8. Organización de un sistema multiprocesador de bus único.

ORGANIZACIÓN SMP

La memoria está organizada de tal manera que se pueden realizar múltiples accesos simultáneos a bloques separados. Dentro de la organización del SMP existen múltiples procesadores, cada uno de los cuales contiene su propia unidad de control, unidad aritmético-lógica y registros. Cada procesador tiene acceso a una memoria principal compartida y dispositivos de E/S a través de algún mecanismo de interconexión; el bus compartido es común a todos los procesadores. Los procesadores se pueden comunicar entre sí a través de la memoria (mensajes e información de estado dejados en espacios de memoria compartidos).

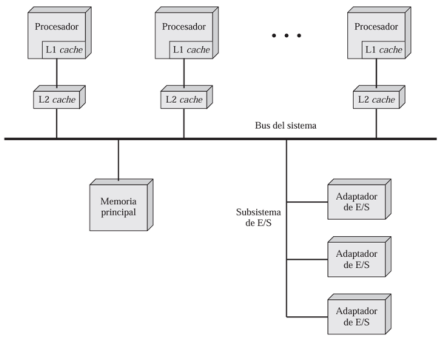


Imagen 9. Estructura de un sistema multiprocesador simétrico con cachés de dos niveles (L1 y L2) y subsistema de E/S.

CONSIDERACIONES DE DISEÑO DE SISTEMAS OPERATIVOS MULTIPROCESADOR

Un sistema operativo SMP gestiona los procesadores y otros recursos del computador, de manera que el usuario puede ver al sistema de la misma forma que si fuera un sistema uniprocesador multiprogramado. Un usuario puede desarrollar aplicaciones que utilicen múltiples procesos o múltiples hilos dentro de procesos sin preocuparse de si estará disponible un único procesador o múltiples procesadores.

Las principales claves de diseño incluyen las siguientes características:

a. **Procesos o hilos simultáneos concurrentes.** Las rutinas del núcleo necesitan ser reentrantes para permitir que varios procesadores ejecuten el mismo código del núcleo simultáneamente. Debido a que múltiples procesadores pueden ejecutar la misma o diferentes partes del código del núcleo, las tablas y la gestión de las estructuras del núcleo deben ser gestionas apropiada mente para impedir interbloqueos u operaciones inválidas.
b. **Planificación.** La planificación se puede realizar por cualquier procesador, por lo que se deben evitar los conflictos. Si se utiliza multihilo a nivel de núcleo, existe la posibilidad de

planificar múltiples hilos del mismo proceso simultáneamente en múltiples procesadores.

c. **Sincronización.** Con múltiples procesos activos, que pueden acceder a espacios de direcciones o recursos compartidos de E/S, se debe tener cuidado en proporcionar una sincronización eficaz. La sincronización es un servicio que fuerza la exclusión mutua y el orden de los eventos.
d. **Gestión de memoria.** La gestión de memoria en un multiprocesador debe tratar con todos los aspectos encontrados en las máquinas uniprocesador. Además, el sistema operativo necesita explotar el paralelismo hardware existente, como las memorias multipuerto, para lograr el mejor rendimiento. Los mecanismos de paginación de los diferentes procesadores deben estar coordinados para asegurar la consistencia cuando varios procesadores comparten una página o segmento y para decidir sobre el reemplazo de una página.
a. **Fiabilidad y tolerancia a fallos.** El sistema operativo no se debe degradar en caso de fallo de un procesador. El planificador y otras partes del sistema operativo deben darse cuenta de la pérdida de un procesador y reestructurar las tablas de gestión apropiadamente.

Micronúcleos

Un micronúcleo es la pequeña parte central de un sistema operativo que proporciona las bases para realizar extensiones modulares. La filosofía fundamental de esta arquitectura consiste en que únicamente las funciones absolutamente esenciales del sistema operativo deben permanecer dentro del núcleo, dejando fuera subsistemas como gestores de archivos o controladores. De esta manera, los

servicios menos esenciales se ejecutan en modo usuario como servidores independientes que se comunican de forma transparente.

Arquitectura micronúcleo

A diferencia de los sistemas monolíticos o por capas donde predomina la estructura vertical y la dependencia de código, la arquitectura de micronúcleo establece una estructura horizontal basada en el modelo cliente/servidor. En esta arquitectura, los componentes externos se implementan como servidores en espacio de usuario que interactúan entre sí mediante el paso de mensajes gestionado por el micronúcleo

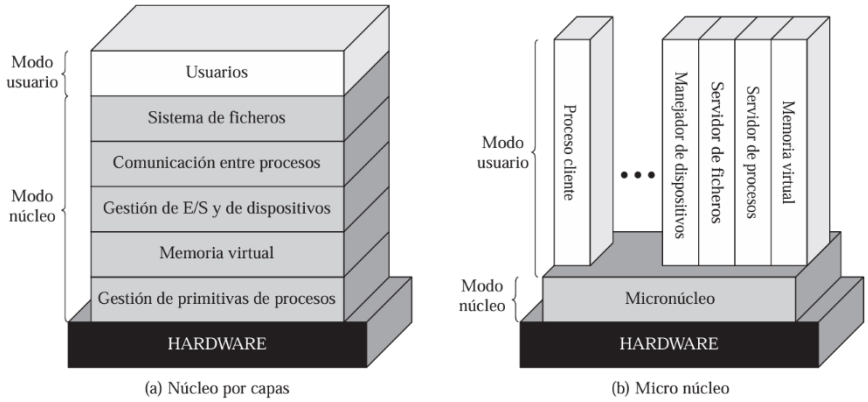


Imagen 10. Comparación entre las arquitecturas de núcleo por capas (monolítico) y micro núcleo (microkernel).

.

Beneficios de una organización micronúcleo

Esta organización proporciona una interfaz uniforme al solicitar todos los servicios por paso de mensajes. Ofrece extensibilidad y flexibilidad para modificar características sin alterar el núcleo. Asimismo, mejora la portabilidad al concentrar el código del procesador en el núcleo, aumenta la fiabilidad al tener un diseño verificable y facilita el soporte de sistemas distribuidos y orientados a objetos.

Rendimiento del micronúcleo

La principal desventaja es el tiempo extra que toma construir, enviar y decodificar mensajes frente a una llamada simple a un servicio. Mientras los micronúcleos de primera generación (como Mach) afectaban la velocidad por ser más grandes y requerir muchas llamadas al sistema, los de segunda generación (como L4) redujeron su tamaño a 12 Kbytes, demostrando ser tan eficientes como los sistemas monolíticos.

Diseño del micronúcleo

El diseño abarca funciones esenciales dependientes del hardware y el soporte a servidores en modo usuario. En gestión de memoria a bajo nivel, mediante las operaciones Grant, Map y Flush, delega la memoria virtual a un paginador externo; como muestra la figura, ante un fallo de página el micronúcleo avisa al paginador, este localiza la página llamando a la función de espacio de direcciones y el micronúcleo reanuda la aplicación. En IPC gestiona mensajes hacia puertos de los procesos, y en E/S convierte las interrupciones hardware en mensajes para los hilos de usuario.

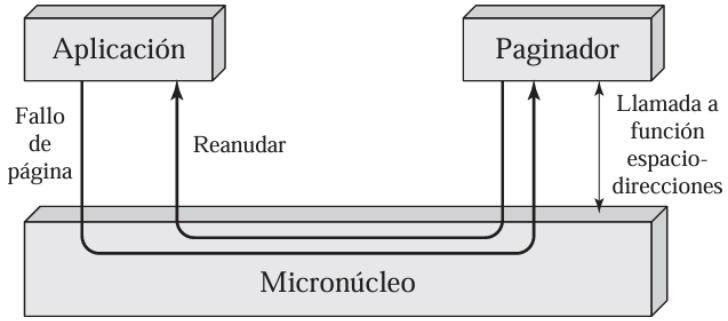


Imagen 11. Mecanismo de gestión de fallos de página mediante IPC en una arquitectura basada en micronúcleo.

Windows

Gestión de Hilos en Windows

Estructura de objetos: Proceso e hilo

Windows implementa procesos e hilos como objetos, siguiendo su arquitectura orientada a objetos. Un proceso es la entidad que corresponde a un trabajo de usuario o aplicación y que posee recursos como memoria y archivos abiertos, mientras que un hilo es la unidad de trabajo real: se activa, ejecuta secuencialmente y es interrumpible, de modo que el procesador puede cambiar a otro hilo en cualquier momento.

Cada proceso Windows debe contener al menos un hilo para ejecutarse, y ese hilo puede a su vez crear otros. En un sistema multiprocesador, varios hilos de un mismo proceso pueden ejecutar en paralelo. A cada proceso se le asigna una ficha (token) de acceso de seguridad que Windows usa para verificar permisos; el proceso también incluye una tabla de manejadores hacia otros objetos que conoce (hilos, archivos, segmentos de memoria compartida, etc.).

Los atributos más relevantes del **objeto proceso** incluyen: ID de proceso, descriptor de seguridad, prioridad base, afinidad de procesador por defecto, límite de cuota, tiempo de ejecución acumulado, contadores de E/S y de operaciones de memoria virtual, puertos de excepción/depurado, y el estado de salida.

Los atributos del **objeto hilo** incluyen: ID de hilo, contexto del hilo (registros y estado volátil), prioridad dinámica y prioridad base, afinidad de procesador del hilo (subconjunto de la del proceso), tiempo de ejecución acumulado, estado de alerta, contador de suspensiones, testigo de personificación, puerto de finalización y estado de salida del hilo.

Multihilo

Windows soporta concurrencia tanto entre procesos como dentro de un mismo proceso: varios hilos de un proceso pueden asignarse a distintos procesadores y ejecutar concurrentemente sin la sobrecarga de usar múltiples

procesos completos. Los hilos de un mismo proceso comparten el espacio de direcciones y los recursos del proceso; los hilos de procesos distintos solo pueden intercambiar información mediante memoria compartida. Un caso típico de aprovechamiento es el proceso servidor multihilo, donde cada petición de un cliente dispara la creación de un nuevo hilo.

Estados de un hilo

- **Listo (ready):** Puede planificarse para ejecución; el micronúcleo lo conoce y lo ordena por prioridad.
- **Substituto (standby):** Seleccionado para ejecutar a continuación en un procesador determinado; si su prioridad es lo bastante alta, puede expulsar al hilo en ejecución.
- **Ejecutando (running):** Corre hasta que es expulsado, agota su porción de tiempo, se bloquea o termina.
- **Esperando (waiting):** Ocurre al bloquearse por un evento (E/S), por sincronización voluntaria, o al ser suspendido por un subsistema.
- **Transición (transition):** Listo para ejecutar, pero a la espera de que un recurso (por ejemplo su pila) esté disponible en memoria.
- **Terminado (terminated):** El hilo finaliza por sí mismo, por otro hilo, o al terminar su proceso padre.

SMP en Windows

Windows soporta una configuración hardware SMP. Los hilos de cualquier proceso, incluyendo los del ejecutivo, pueden ejecutar en cualquier procesador. En ausencia de restricciones de afinidad, explicadas en el siguiente párrafo, el micronúcleo asigna un hilo listo al siguiente procesador disponible. Esto asegura que ningún procesador está ocioso o está ejecutando un hilo de menor prioridad cuando un hilo de mayor prioridad está listo. Múltiples hilos de un proceso pueden ejecutar a la vez en múltiples procesadores.

Por defecto, el micronúcleo utiliza la política **afinidad débil** (soft affinity) para asignar procesadores a los hilos: el planificador intenta asignar un proceso listo al mismo procesador que lo ejecutó la última vez. Esto ayuda a reutilizar datos que estén todavía en la memoria cache del procesador de la ejecución previa del hilo. Para una aplicación es posible restringir la ejecución de sus hilos a determinados procesadores **afinidad fuerte** (hard affinity).

Linux

Linux es un sistema operativo que se define por tener código abierto, esto significa que cualquiera puede ver, modificar y compartirlo de forma completamente gratuita. Hasta ahora ha hecho una penetración significativa en el mundo corporativo, esto no deriva solo de que sea un software libre, sino también a la calidad del núcleo de Linux, ya que es muy modular y fácilmente configurable, resulta óptimo para incrementar el rendimiento de una variedad de plataformas hardware.



Imagen 12. Sistema Operativo Linux.

Gestión de Hilos en Linux

Un hilo en un sistema operativo es aquel que se encarga del flujo de ejecución dentro de un proceso, supongamos que un programa que descarga un archivo y muestra una barra de progreso de la descarga, este descarga y archivo, actualiza el progreso y al final responde al usuario, con un solo hilo esta tarea tardaría mucho ya que si la descarga se detiene todo lo demás también y el programa podría quedarse congelado. Con varios hilos las tareas podrán avanzar de manera eficiente y rápida ya que se le asignaría un hilo a cada tarea. Un ejemplo claro seria al utilizar una aplicación como Word este es un procesador de texto, se podrían realizar de manera concurrente tareas como en el hilo 1; se encargaría de detectar lo que se esta escribiendo con el teclado y mostrar las letras en pantalla, el hilo 2; revisa la ortografía y gramática en tiempo real en segundo plano y el hilo 3;

hace el guardado automático del archivo cada cierto minuto sin interrumpir al usuario. Son útiles para multitareas en un mismo programa, así mismo mayor eficiencia y rapidez.

Ahora en Linux un proceso o tarea se representa por una estructura de datos llamada task_struct, que contiene información de varias categorías como:

- Estado: Indica en que estado se encuentra la ejecución de un proceso (ejecutado, listo, suspendido, detenido, zombi).
- Información de planificación: esta es una información solicitada por linux para planificar procesos ya sea un proceso normal o de tiempo real y tener de esta manera una prioridad.
- Identificadores: Cada proceso tiene un identificador único dentro de un proceso, también tiene identificadores de usuario y grupo.

Los sistemas UNIX tradicionales estos soportan un solo hilo por proceso, mientras que los sistemas UNIX modernos ya suelen proporcionar soporte para múltiples hilos de un nivel de núcleo por proceso. Linux proporciona en sus versiones modernas una solución única en la que no se diferencia entre hilos y proceso. Así mismo utilizando un mecanismo similar al de los procesos ligeros de Solaris.

En Linux se crea un proceso copiando los atributos del proceso actual. Es decir, un proceso se puede clonar de forma que comparten recursos, archivos, manejadores de señales y memoria virtual. En lugar del mandato normal fork(), los procesos que se crean usando el mandato clone (). Este dicho mandato incluye un conjunto de flags como argumentos. Clone () es utilizado para clonar, crea una nueva tarea y dependiendo de si comparten los recursos con la tarea original y así formar el hilo del proceso. Fork() y clone() se conectan de forma que si se utiliza clone() activando las flags para compartir todo, se crea un hilo, mientras que si se usa clone() sin activar ninguna flag de compartición, se comporta igual que un fork() tradicional creando un proceso de hilo aislado.

SMP en Linux

Linux está diseñado para funcionar en arquitecturas de multiprocesamiento simétrico, lo que significa que cualquier hilo (incluyendo los que ejecutan código del propio núcleo) puede planificarse en cualquiera de los procesadores disponibles. Esto obliga a que las estructuras de datos internas del núcleo (tablas de procesos, colas de planificación, buffers, entre otras) queden protegidas frente al acceso simultáneo de varios procesadores.

Para resolver esto, el mecanismo más utilizado dentro del núcleo de Linux para proteger una sección crítica es el cerrojo cíclico o spinlock. Su funcionamiento es sencillo: un hilo que intenta entrar a la sección crítica revisa una posición de memoria asociada al cerrojo; si está libre, la toma y continúa, y si está ocupada, se queda comprobando el valor repetidamente (en espera activa) hasta que el cerrojo se libere. Esta espera activa resulta conveniente porque, dentro del núcleo, las secciones críticas suelen ser muy breves (normalmente más cortas que el tiempo que tomaría un cambio de contexto), por lo que resulta más eficiente que bloquear al hilo por completo. Existen distintas variantes del cerrojo cíclico según si es necesario además deshabilitar interrupciones locales o rutinas de "mitad inferior" mientras se mantiene el cerrojo, y también una versión de lectura-escritura que permite que varios hilos lean una misma estructura al mismo tiempo, reservando el acceso exclusivo solo para cuando alguno necesita escribir en ella.

Un detalle particular de la implementación de Linux es que el cerrojo cíclico se comporta de forma distinta según el hardware: en un sistema con un solo procesador, el spinlock ni siquiera necesita comprobar la memoria compartida, ya que basta con habilitar o deshabilitar la expulsión del núcleo para lograr el mismo efecto de exclusión; en un sistema multiprocesador, en cambio, el spinlock sí se compila para realizar la comprobación real de la posición de memoria, que es lo que efectivamente evita que dos procesadores modifiquen la misma estructura al mismo tiempo. Esto permite que el mismo código del núcleo funcione igual sin importar si se ejecuta sobre uno o varios procesadores.

UNIX/Solaris

Gestión de Hilos en UNIX/Solaris

En las versiones modernas de UNIX, la estructura y gestión de hilos se diseña con el objetivo de administrar la concurrencia de forma eficiente, separando la propiedad de los recursos de la ejecución del flujo de instrucciones. Para lograr esto, la arquitectura del sistema se divide en cuatro conceptos fundamentales.

El primero de ellos es el proceso, que representa la estructura clásica que incluye el espacio de direcciones virtuales y los recursos del sistema asociados a un programa en ejecución. Dentro de este espacio, la aplicación tiene la capacidad de crear hilos de nivel de usuario mediante una biblioteca específica. Dado que el núcleo del sistema operativo no interviene directamente en la gestión de estos hilos, su creación y eliminación resultan ser operaciones rápidas.

El vínculo entre estos hilos de usuario y el sistema operativo se establece a través del tercer concepto: el proceso ligero o LWP. Esta estructura, reside dentro del núcleo que actúa como un procesador virtual sobre el cual se multiplexan los hilos de nivel de usuario. Finalmente, cada proceso ligero está enlazado a un hilo de núcleo, constituyendo la verdadera entidad que el sistema operativo reconoce, planifica y despacha para su ejecución en el hardware físico.

Esta organización facilita la administración de la concurrencia y optimiza el uso de los recursos del sistema, permitiendo que diversas tareas se ejecuten de manera organizada. La gestión operativa se fundamenta en un mapeo mediante el cual los hilos de las aplicaciones se asocian a los procesos ligeros disponibles, y estos, a su vez, se traducen en hilos de núcleo.

Gracias a este diseño, el programador dispone de la flexibilidad necesaria para generar una gran cantidad de hilos de usuario sin sobrecargar el equipo, ya que el núcleo se limita exclusivamente a planificar y administrar los hilos de núcleo que se encuentran activos.

SMP en UNIX/Solaris

Los sistemas UNIX tradicionales se concibieron para funcionar sobre un único procesador. Por eso, su núcleo carecía de mecanismos para proteger las estructuras de datos internas cuando varios procesadores intentaban acceder a ellas al mismo tiempo — una limitación crítica en cualquier arquitectura multiprocesador. A esto se sumaba un diseño poco flexible: un solo tipo de sistema de archivos, una sola política de planificación, y un núcleo cada vez más grande y difícil de extender a medida que se le añadían funciones nuevas.

Con la llegada de versiones modernas de UNIX, este panorama cambió. Solaris, la variante de Sun basada en SVR4, es el ejemplo más representativo: incorpora un núcleo completamente expulsivo con soporte total para multiprocesamiento simétrico (SMP), lo que le permite repartir el trabajo del sistema entre múltiples procesadores de forma segura y eficiente.

Uno de los retos técnicos más importantes al diseñar un núcleo para SMP es la protección de datos compartidos frente a las interrupciones. En un sistema de un solo procesador, basta con subir temporalmente el nivel de prioridad para bloquear interrupciones mientras se accede a una estructura crítica. Pero en un entorno multiprocesador esto se vuelve costoso: el núcleo tendría que bloquear las interrupciones en todos los procesadores a la vez para garantizar la protección de un mismo dato.

Solaris resuelve este problema convirtiendo las interrupciones en hilos de núcleo propios, cada uno con su identificador, prioridad, contexto y pila. En lugar de deshabilitar interrupciones a nivel de hardware en todo el sistema, sincroniza estos hilos usando las mismas primitivas de exclusión mutua que emplea para cualquier otro hilo del núcleo, y les asigna la prioridad más alta del sistema. Cuando llega una interrupción, el procesador que la recibe "inmoviliza" temporalmente el hilo que estaba ejecutando (sin poder migrarlo a otro CPU) y pasa a ejecutar el hilo de interrupción correspondiente. Las pruebas realizadas con este enfoque demostraron un rendimiento superior frente a las estrategias tradicionales de manejo de interrupciones.

Investigación de Concurrencia

Definición de concurrencia

La concurrencia es uno de los temas centrales del diseño de todo sistema operativo, ya sea monoprocesador, multiprocesador o distribuido. Surge de una característica básica de los sistemas multiprogramados: no puede predecirse la velocidad relativa de ejecución de los procesos, porque esta depende de la actividad de los demás procesos, de cómo el sistema operativo maneja las interrupciones y de las políticas de planificación vigentes. Este concepto abarca varios aspectos relacionados entre sí: la comunicación entre procesos, la compartición de recursos, la sincronización de actividades de múltiples procesos y la repartición del tiempo de procesador entre ellos.

Esta noción no depende de tener múltiples procesadores físicos: los mismos problemas de fondo aparecen tanto si los procesos se entrelazan en el tiempo dentro de un único procesador (dando la apariencia de ejecución simultánea sin lograr paralelismo real) como si se solapan genuinamente en un sistema con varios procesadores. Aunque a primera vista el entrelazado y el solapamiento parecen modos de ejecución fundamentalmente distintos, ambos pueden entenderse como ejemplos de procesamiento concurrente, y ambos plantean exactamente los mismos problemas: peligros al compartir recursos globales, dificultad para que el sistema operativo asigne recursos de forma óptima, y errores de programación difíciles de localizar por ser típicamente no deterministas y no reproducibles.

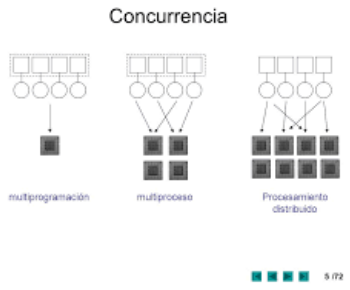


Imagen 13. Diferentes niveles de concurrencia: multiprogramación, multiproceso y procesamiento distribuido.

Tipos de concurrencia

Se puede clasificar desde dos ángulos complementarios: según el grado real de simultaneidad que se logra, y según los contextos en los que aparece dentro del diseño de un sistema.

Según el grado de simultaneidad: concurrencia aparente y concurrencia real

En un sistema multiprogramado con una única UCP, varios procesos se intercalan en el tiempo repartiéndose el procesador, de modo que dan la impresión de ejecutarse simultáneamente sin que eso ocurra en realidad. En un sistema multiprocesador, en cambio, es posible que se ejecuten tantos procesos de forma verdaderamente simultánea como procesadores haya disponibles. De ahí se desprende una distinción clara:

- **Concurrencia aparente:** se produce siempre que el número de procesos sea mayor que el de procesadores disponibles, es decir, cuando hay más de un proceso por procesador y estos deben repartirse el tiempo de CPU mediante intercalado.
- **Concurrencia real:** se produce cuando hay un proceso por procesador, de modo que la ejecución simultánea es efectiva y no solo simulada.

Esta distinción se sostiene tanto en arquitecturas de multiprocesador (memoria compartida entre varios procesadores) como de multicomputadora (memoria distribuida entre nodos completos que se comunican mediante paso de mensajes a través de una red de interconexión); en ambos casos es posible la ejecución simultánea real de procesos sobre distintos procesadores.

Según el contexto de diseño

La concurrencia aparece en al menos tres contextos distintos dentro de un sistema:

- **Múltiples aplicaciones:** la multiprogramación fue ideada precisamente para permitir compartir dinámicamente el tiempo de procesamiento entre varias aplicaciones activas a la vez.
- **Aplicaciones estructuradas:** como extensión de los principios del diseño modular y la programación estructurada, ciertas aplicaciones se programan eficazmente como un conjunto de procesos concurrentes que cooperan entre sí.
- **Estructura del propio sistema operativo:** los sistemas operativos suelen implementarse a sí mismos como un conjunto de procesos o hilos que se ejecutan de forma concurrente, aplicando las mismas ventajas constructivas que en el software de aplicación.

Estos contextos, a su vez, se relacionan con tres formas de gestionar procesos que dependen del tipo de arquitectura subyacente: la multiprogramación (gestión de múltiples procesos dentro de un sistema monoprocesador), el multiprocesamiento (gestión de múltiples procesos dentro de un multiprocesador) y el procesamiento distribuido (gestión de múltiples procesos que se ejecutan sobre múltiples sistemas de cómputo distribuidos).

Según su origen: paralelismo explícito e implícito

Una clasificación complementaria distingue el paralelismo según quién lo determina dentro del programa:

- **Paralelismo explícito:** el programador lo indica de forma específica mediante una construcción de concurrencia, típicamente del tipo cobegin ... coend, dentro de la cual cada proposición puede asignarse a un procesador distinto. Es susceptible de errores de programación difíciles de detectar y depurar, y el programador puede omitir situaciones donde el paralelismo sería aplicable.
- **Paralelismo implícito:** es el paralelismo intrínseco del propio algoritmo, no establecido de forma explícita por el programador, sino detectado automáticamente por el compilador o el hardware, por ejemplo, mediante técnicas de distribución de ciclos o reducción de la altura del árbol de evaluación de expresiones.

Modelos de programación concurrente

Semáforos

El primer avance fundamental en el tratamiento de los problemas de programación concurrente ocurre en 1965 con el tratado de Dijkstra. Dijkstra estaba involucrado en el diseño de un sistema operativo como una colección de procesos secuenciales cooperantes y con el desarrollo de mecanismos eficientes y fiables para dar soporte a la cooperación. Estos mecanismos podrían ser usados fácilmente por los procesos de usuario si el procesador y el sistema operativo colaborasen en hacerlos disponibles.

El principio fundamental es éste: dos o más procesos pueden cooperar por medio de simples señales, tales que un proceso pueda ser obligado a parar en un lugar específico hasta que haya recibido una señal específica. Cualquier requisito complejo de coordinación puede ser satisfecho con la estructura de señales apropiada. Para la señalización, se utilizan unas variables especiales llamadas semáforos. Para transmitir una señal vía el semáforo *s*, el proceso ejecutará la primitiva *semSignal(s)*. Para recibir una señal vía el semáforo *s*, el proceso ejecutará la primitiva *semWait(s)*; si la correspondiente señal no se ha transmitido todavía, el proceso se suspenderá hasta que la transmisión tenga lugar.

Para conseguir el efecto deseado, el semáforo puede ser visto como una variable que contiene un valor entero sobre el cual sólo están definidas tres operaciones:

1. Un semáforo puede ser inicializado a un valor no negativo.
2. La operación *semWait* decrementa el valor del semáforo. Si el valor pasa a ser negativo, entonces el proceso que está ejecutando *semWait* se bloquea. En otro caso, el proceso continúa su ejecución.
3. La operación *semSignal* incrementa el valor del semáforo. Si el valor es menor o igual que cero, entonces se desbloquea uno de los procesos bloqueados en la operación *semWait*.

Aparte de estas tres operaciones no hay manera de inspeccionar o manipular un semáforo.

Con esta definición básica establecida, resulta necesario preguntarse cómo se implementan realmente estas tres operaciones y qué ocurre internamente cuando un proceso ejecuta *semWait* o *semSignal*. La estructura de datos que representa a un semáforo contiene, en esencia, dos elementos: un contador entero que refleja su estado actual, y una cola donde se van almacenando los procesos que quedan bloqueados en espera de una señal. Cuando un proceso ejecuta *semWait* sobre un semáforo cuyo contador se vuelve negativo tras la operación, dicho proceso no simplemente "espera" de forma pasiva verificando continuamente el valor del semáforo (lo que se conoce como espera activa y desperdicia ciclos de procesador); en su lugar, es colocado en la cola de espera del semáforo y su ejecución se suspende por completo, cediendo el procesador a otros procesos.

Solo cuando otro proceso ejecute *semSignal* sobre ese mismo semáforo será extraído de la cola y devuelto a la lista de procesos listos para ejecutar. Es importante señalar que ambas primitivas, *semWait* y *semSignal*, se consideran operaciones atómicas: esto significa que su ejecución no puede ser interrumpida a medias por ningún otro proceso, garantizando así que no se produzcan condiciones de carrera al momento de decrementar o incrementar el valor del semáforo.

A partir de esta definición general surge una variante más restringida y sencilla, conocida como semáforo binario o mutex. A diferencia del semáforo general (también llamado semáforo con contador), que puede tomar cualquier valor entero, el semáforo binario únicamente puede alternar entre los valores 0 y 1, comportándose de manera muy similar a un simple candado: cuando vale 1, el recurso está disponible y cualquier proceso puede tomarlo (poniendo el valor en 0); cuando vale 0, el recurso está ocupado y cualquier proceso que intente acceder a él quedará bloqueado hasta que sea liberado.

Aunque en principio esta versión resulta más sencilla de implementar que el semáforo general, ambas tienen exactamente la misma capacidad expresiva, es decir, cualquier problema de sincronización que pueda resolverse con un semáforo general puede resolverse igualmente con uno binario, y viceversa.

Otro aspecto relevante en el funcionamiento interno de los semáforos tiene que ver con el orden en que se atiende a los procesos que se van acumulando en la cola de espera. Cuando varios procesos quedan bloqueados simultáneamente esperando la misma señal, el sistema debe decidir a cuál de ellos despertar primero cuando finalmente se ejecute un *semSignal*.

Si esta política de selección respeta un orden estrictamente FIFO, es decir, atiende primero al proceso que lleva más tiempo esperando, se dice que se trata de un semáforo fuerte, y este tipo de implementación tiene la ventaja de garantizar que ningún proceso quedará esperando indefinidamente mientras otros procesos más "afortunados" son atendidos una y otra vez (evitando así el problema de la inanición). Por el contrario, si no se establece ningún criterio particular sobre el orden de salida de la cola, se habla de un semáforo débil, en el cual sí existe el riesgo de que algún proceso quede postergado de manera indefinida.

Dado que los sistemas operativos reales suelen implementar semáforos fuertes por ser más convenientes y predecibles, este es el modelo que normalmente se asume al analizar algoritmos basados en semáforos.

Los semáforos pueden utilizarse para resolver la exclusión mutua. La idea consiste en asociar un semáforo a cada recurso compartido que deba protegerse, inicializándolo en 1 para indicar que dicho recurso está disponible.

Cada proceso que desee acceder a su sección crítica, es decir, a la porción de código que manipula el recurso compartido, deberá ejecutar primero *semWait* sobre ese semáforo. El primer proceso en hacerlo encontrará el semáforo en 1, por lo que podrá continuar de inmediato hacia su sección crítica, dejando el semáforo en 0. Cualquier otro proceso que intente entrar mientras el recurso sigue ocupado se topará con un semáforo en 0 o negativo, quedando automáticamente bloqueado. Solo cuando el proceso que originalmente obtuvo el acceso termine su sección crítica y ejecute *semSignal*, el semáforo volverá a incrementarse, liberando a uno de los procesos en espera para que pueda proceder.

Finalmente, uno de los ejemplos más representativos e ilustrativos del uso práctico de los semáforos es el llamado problema del productor/consumidor, un escenario recurrente en programación concurrente donde uno o varios procesos productores generan datos que van colocando en un buffer compartido, mientras que un proceso consumidor los va retirando de ese mismo buffer.

El reto central consiste en evitar dos situaciones problemáticas: que el productor intente escribir en un buffer completamente lleno, o que el consumidor intente leer de un buffer vacío. Para resolver esto de manera elegante, suelen emplearse dos semáforos con propósitos distintos: uno se encarga exclusivamente de garantizar la exclusión mutua sobre el acceso al buffer (para que productor y consumidor nunca lo manipulen al mismo tiempo), mientras que un segundo semáforo funciona como mecanismo de sincronización, obligando al consumidor a esperar cuando no hay datos disponibles para consumir.

Monitores

Aunque los semáforos proporcionan una herramienta potente y flexible para conseguir la exclusión mutua y la coordinación de procesos, en la práctica puede resultar difícil producir un programa correcto utilizando este mecanismo. La dificultad radica en que las operaciones *semWait* y *semSignal* suelen estar dispersas a lo largo de todo el programa, lo que dificulta ver con claridad el efecto global que tienen sobre los semáforos a los que afectan; basta con que el programador olvide o coloque incorrectamente una sola de estas operaciones para que todo el esquema de sincronización falle.

Como respuesta a este problema surge el monitor, una construcción propia de los lenguajes de programación que ofrece una funcionalidad equivalente a la de los semáforos, pero resulta considerablemente más fácil de controlar. El concepto fue definido formalmente por primera vez por Hoare en 1974, y desde entonces ha sido implementado en distintos lenguajes de programación, como Pascal Concurrente, Pascal-Plus, Modula-2, Modula-3 y Java, además de haberse desarrollado también como biblioteca de programa, lo cual permite a los programadores colocar cerrojos monitor sobre cualquier objeto que lo requiera.

Un monitor es, en esencia, un módulo software compuesto por uno o más procedimientos, una secuencia de inicialización y datos locales. Sus características principales son tres: primero, las variables de datos locales solo pueden ser accedidas por los procedimientos internos del propio monitor, nunca desde el exterior; segundo, un proceso entra en el monitor invocando alguno de sus procedimientos; y tercero, solo un proceso puede estar ejecutando dentro del monitor en un momento dado, de modo que cualquier otro proceso que intente invocarlo quedará bloqueado hasta que el monitor quede disponible nuevamente.

Estas dos primeras características guardan bastante semejanza con las de los objetos en la programación orientada a objetos, por lo que en un sistema operativo o lenguaje orientado a objetos un monitor puede implementarse directamente como un objeto con propiedades especiales. Precisamente por respetar la regla de "un solo proceso a la vez", el monitor logra proporcionar

exclusión mutua de forma prácticamente automática: basta con colocar una estructura de datos compartida dentro del monitor para que quede protegida, ya que solo un proceso podrá acceder a ella en cada instante.

Sin embargo, para que el monitor sea realmente útil en programación concurrente, no basta con la exclusión mutua; también necesita mecanismos de sincronización. Considérese, por ejemplo, un proceso que invoca al monitor y que, mientras está dentro, necesita bloquearse hasta que se cumpla cierta condición. Aquí surge un requisito delicado: el proceso no solo debe bloquearse, sino que además debe liberar el monitor para que otro proceso pueda entrar en él; más adelante, cuando la condición se cumpla y el monitor quede libre, el proceso original deberá poder retomar su ejecución exactamente en el punto donde se quedó.

Esta necesidad se resuelve mediante las llamadas variables condición, un tipo de dato especial contenido dentro del monitor y accesible únicamente desde dentro de él, que se manipulan mediante dos funciones:

- *cwait(c)*: suspende la ejecución del proceso que la invoca sobre la condición c, dejando el monitor disponible para que otro proceso pueda usarlo.
- *csignal(c)*: retoma la ejecución de algún proceso que estuviera bloqueado por un *cwait* en esa misma condición; si hay varios procesos esperando, elige a uno de ellos, y si no hay ninguno, simplemente no hace nada.

Es importante notar que estas operaciones *wait/signal* de los monitores no funcionan igual que las de los semáforos: si un proceso dentro del monitor ejecuta un *csignal* y no hay ningún proceso esperando en esa variable condición, la señal simplemente se pierde, sin generar ningún efecto posterior ni quedar "almacenada" para el futuro.

La definición original de Hoare exige que, si hay al menos un proceso esperando en la cola de una condición, dicho proceso se ejecute inmediatamente en cuanto otro proceso realice un *csignal* sobre esa condición. Esto genera dos

desventajas prácticas: primero, si el proceso que emite la señal aún no ha terminado con el monitor, se requieren dos cambios de proceso adicionales (uno para bloquearlo y otro para reactivarlo posteriormente); y segundo, la planificación asociada a la señal debe ser perfectamente fiable, ya que si algún otro proceso lograra colarse en el monitor antes de que el proceso señalado sea activado, la condición bajo la cual fue despertado podría haber cambiado, dejándolo colgado indefinidamente.

Para resolver estos problemas, Lampson y Redell desarrollaron una definición alternativa de monitor para el lenguaje Mesa (adoptada también en Modula-3), en la cual la primitiva *csignal* se sustituye por *cnotify*. Cuando un proceso ejecuta *cnotify(x)*, simplemente provoca que la cola de la condición x sea notificada, pero el proceso que emitió la señal continúa ejecutando con normalidad. El proceso que encabeza la cola de la condición será retomado en algún momento futuro conveniente, cuando el monitor quede libre; sin embargo, como no hay garantía de que ningún otro proceso entre antes que él, el proceso notificado debe volver a comprobar la condición por sí mismo (por eso el código pasa de usar sentencias if a bucles while).

Esta variante también introduce una mejora adicional: la posibilidad de asociar un temporizador a cada condición, de modo que un proceso que lleve esperando demasiado tiempo sea puesto en estado Listo automáticamente, evitando así la inanición indefinida en caso de que otro proceso falle antes de señalar la condición. Además, incorpora la primitiva *cbroadcast*, que provoca que todos los procesos esperando en una condición pasen a estado Listo simultáneamente, resultando útil en situaciones donde un proceso no puede saber con precisión cuántos otros procesos deben reactivarse (por ejemplo, en la gestión de un buffer de memoria de tamaño variable).

Entre las principales ventajas del modelo de Lampson/Redell frente al de Hoare destacan dos: es menos propenso a errores, ya que al usar bucles while cada procedimiento vuelve a verificar su condición tras ser notificado, por lo que una señal o difusión incorrecta no provoca directamente un fallo en el programa; y además se

presta a un enfoque más modular en la construcción de programas, evitando que el programador tenga que reprogramar todos los procesos que señalizan cada vez que cambia ligeramente una condición.

Paso de mensajes

Cuando los procesos interaccionan entre sí, deben satisfacerse dos requisitos fundamentales: sincronización y comunicación. Por un lado, los procesos necesitan sincronizarse para conseguir exclusión mutua; por otro, los procesos cooperantes pueden necesitar intercambiar información entre sí. El paso de mensajes es un enfoque que resulta especialmente interesante porque proporciona ambas funciones a la vez, con la ventaja añadida de que se presta a ser implementado tanto en sistemas distribuidos como en multiprocesadores de memoria compartida e incluso en sistemas monoprocesador, lo cual lo convierte en un mecanismo mucho más versátil que los semáforos o los monitores en cuanto a su rango de aplicación.

Los sistemas de paso de mensajes se presentan en muchas modalidades distintas según la implementación concreta, pero en su forma más básica, la funcionalidad se proporciona mediante un par de primitivas: *send(destino, mensaje)* y *receive(origen, mensaje)*. Este es el conjunto mínimo de operaciones necesarias para que dos procesos puedan entablar comunicación: el proceso emisor envía información en forma de mensaje a otro proceso identificado por el parámetro destino, mientras que el proceso receptor obtiene esa información ejecutando la primitiva *receive*, indicando de dónde proviene el mensaje.

A partir de esta base mínima, existen varias decisiones de diseño que determinan el comportamiento concreto de un sistema de paso de mensajes: la forma de sincronización entre emisor y receptor, el esquema de direccionamiento utilizado, el formato del mensaje y la disciplina de cola aplicada.

La comunicación de un mensaje entre dos procesos implica cierto nivel de sincronización inherente entre ambos, ya que el receptor no puede recibir un

mensaje hasta que otro proceso lo haya enviado previamente. Por ello resulta necesario especificar qué le sucede exactamente a un proceso después de ejecutar una primitiva *send* o *receive.*

En el caso de *send*, existen dos posibilidades: el proceso emisor puede bloquearse hasta que el mensaje sea efectivamente recibido, o bien puede continuar su ejecución sin esperar. De manera similar, cuando un proceso ejecuta *receive*, si el mensaje ya fue enviado previamente, lo recibe de inmediato y continúa su ejecución; pero si no hay ningún mensaje esperando, el proceso puede optar por bloquearse hasta que el mensaje llegue, o bien continuar ejecutando, abandonando el intento de recepción.

Combinando estas posibilidades surgen tres esquemas típicos, aunque un sistema concreto normalmente solo implementa uno o dos de ellos:

- Envío bloqueante, recepción bloqueante: tanto el emisor como el receptor se bloquean hasta que el mensaje se entrega por completo; a este esquema también se le conoce como *rendezvous*, ya que ambas partes deben "encontrarse" en el tiempo para que la comunicación se complete.
- Envío no bloqueante, recepción bloqueante: el emisor puede continuar de inmediato con su ejecución, pero el receptor sí se bloqueará hasta que llegue el mensaje solicitado. Esta combinación es probablemente la más útil en la práctica.
- Envío no bloqueante, recepción no bloqueante: ninguna de las dos partes tiene que esperar en ningún momento.

El intercambio de mensajes en la programación concurrente se basa en las primitivas de envío y recepción, cuyas modalidades determinan la eficiencia y el control de los procesos. El envío no bloqueante permite que el emisor continúe inmediatamente con su trabajo tras enviar un mensaje, lo cual resulta ideal para operaciones como la impresión; sin embargo, conlleva el riesgo de que un error de programación sature el sistema con mensajes infinitos y traslada al programador la responsabilidad de verificar su recepción mediante confirmaciones. Por su parte, la

recepción bloqueante es la más natural porque el proceso receptor espera la información necesaria para avanzar, aunque corre el riesgo de un bloqueo indefinido si el emisor falla o el mensaje se pierde. Usar una recepción no bloqueante mitiga este riesgo, pero introduce la posibilidad de perder mensajes si estos se envían justo después de que el receptor haya ejecutado la comprobación.

Para que estas operaciones funcionen, es indispensable un mecanismo de direccionamiento que especifique el origen y destino. El direccionamiento directo incluye el identificador exacto del destinatario en la primitiva de envío, permitiendo que la recepción sea explícita al buscar un emisor específico o implícita al aceptar mensajes de cualquier origen, lo cual es útil para procesos servidores como los de impresión. En contraste, el direccionamiento indirecto utiliza buzones o puertos compartidos donde se depositan los mensajes, logrando desacoplar al emisor del receptor y ofreciendo una gran flexibilidad mediante relaciones que pueden ser uno-a-uno, muchos-a-uno, uno-a-muchos o muchos-a-muchos de forma estática o dinámica.

En cuanto al manejo de la información, el formato del mensaje puede ser de longitud fija para minimizar sobrecargas o de longitud variable, dividiéndose en una cabecera con datos de control, tipos y prioridades, y un cuerpo con el contenido real. Para organizar estos mensajes en las colas, la disciplina más sencilla es FIFO, aunque es posible incorporar prioridades o permitir que el receptor seleccione el mensaje de su interés. Finalmente, estas bases permiten resolver problemas clásicos de concurrencia: la exclusión mutua se puede lograr compartiendo un buzón con un único mensaje nulo que actúa como testigo para acceder a la sección crítica, mientras que el problema del productor/consumidor explicado anteriormente se gestiona con eficacia mediante dos buzones, donde uno almacena los datos generados y el otro controla el flujo del buffer mediante mensajes nulos.

Memoria compartida

En el modelo de memoria compartida, la comunicación entre procesos no ocurre mediante el intercambio explícito de información, sino a través del acceso

directo a una región de memoria que todos los procesos cooperantes pueden leer y escribir. Para que esto sea posible, primero debe establecerse un segmento de memoria compartida: normalmente reside en el espacio de direcciones del proceso que lo crea, y cualquier otro proceso que quiera comunicarse a través de él debe conectarse explícitamente a ese espacio de direcciones, ya que el sistema operativo, por defecto, protege la memoria de un proceso frente al acceso de cualquier otro. Una vez establecida esta conexión, todos los accesos posteriores se comportan como accesos ordinarios a memoria, sin necesidad de que intervenga el kernel ni de realizar llamadas al sistema adicionales, lo cual es precisamente la razón de que este modelo sea más rápido que el paso de mensajes en la práctica.

Sin embargo, esta velocidad tiene un costo: como el sistema operativo no arbitra ni conoce el significado de lo que se escribe en la región compartida, son los propios procesos quienes deben ponerse de acuerdo en el formato y la ubicación de los datos, y quienes deben garantizar que nunca dos de ellos escriban en la misma posición de manera simultánea. Esta responsabilidad recae en mecanismos explícitos de sincronización: semáforos, exclusión mutua mediante locks, monitores, que deben aplicarse de forma disciplinada en cada sección crítica del programa; si un solo proceso olvida aplicar correctamente el protocolo de sincronización, toda la información compartida puede corromperse sin que exista ningún aviso automático del sistema.

Un ejemplo clásico usado para ilustrar el modelo es el problema del productor-consumidor, ejemplo utilizado con el semáforo.

Modelo de Actores

El modelo de actores parte de una premisa radicalmente distinta a la memoria compartida: elimina por completo la posibilidad de que dos unidades de ejecución accedan al mismo estado. Un actor es, a la vez, una unidad de estado y una unidad de comportamiento: encapsula datos privados que ningún otro actor puede leer ni modificar directamente, y solo puede ser influido desde el exterior mediante mensajes asíncronos que llegan a una cola de entrada (comúnmente llamada

mailbox) asociada exclusivamente a él. Cada actor procesa los mensajes de su cola de uno en uno y de forma estrictamente secuencial, lo que significa que, dentro de un mismo actor, nunca hay dos fragmentos de código ejecutándose a la vez sobre el mismo estado; toda la concurrencia del sistema surge, en cambio, de tener muchísimos actores ejecutándose en paralelo, cada uno atendiendo su propia cola de forma independiente.

Al recibir un mensaje, un actor solo puede hacer tres cosas: modificar su propio estado interno, enviar mensajes a otros actores (a los que conoce por referencia, no por acceso directo a su memoria) o crear nuevos actores hijos. Esta restricción tan estricta es justamente lo que le da al modelo sus garantías: como no existe memoria mutable compartida entre actores, desaparecen por diseño tanto las condiciones de carrera como los interbloqueos clásicos asociados a locks, y el razonamiento sobre cada actor puede hacerse de forma casi tan simple como el de una función pura, ya que su comportamiento depende únicamente de su estado interno y del mensaje recibido.

Otro rasgo distintivo del modelo es su forma de manejar errores: en lugar de intentar prevenir toda falla, se asume que los actores fallarán de vez en cuando por causas transitorias, y en vez de propagar ese fallo a todo el sistema, se opta por un manejo "optimista" en el que el actor problemático simplemente se reinicia (a menudo bajo la supervisión de un actor "padre" que forma parte de una jerarquía de supervisión), mientras el resto del sistema continúa funcionando con normalidad. Esta filosofía —popularizada por Erlang, diseñado originalmente por Ericsson para sistemas de telecomunicaciones que debían tolerar fallos sin caerse por completo— es lo que hace del modelo de actores una opción tan usada en sistemas distribuidos, servidores de alta disponibilidad y frameworks como Akka sobre la JVM.

Procesos secuenciales comunicantes

CSP nace, a diferencia de los otros modelos, como una teoría matemática formal antes que como una técnica de implementación: su objetivo original era proporcionar una notación rigurosa para especificar y verificar el comportamiento de

sistemas concurrentes, permitiendo detectar problemas como interbloqueos o livelocks antes incluso de escribir una sola línea de código real. En este modelo, un sistema concurrente se describe como un conjunto de procesos completamente independientes, cada uno con su propia secuencia interna de instrucciones, que jamás comparten memoria entre sí y cuya única forma de interactuar es a través de canales de comunicación explícitos y unidireccionales que conectan a pares (o grupos) de procesos.

La diferencia clave frente al paso de mensajes tradicional está en la naturaleza de la sincronización: en CSP, tanto el envío como la recepción a través de un canal son operaciones estrictamente bloqueantes y deben coincidir en el tiempo, de modo que un proceso que intenta escribir en un canal se queda esperando hasta que otro proceso, en el otro extremo, esté simultáneamente listo para leer. No existe, por tanto, una cola intermedia donde los mensajes puedan acumularse mientras espera a ser recibidos: la propia comunicación es el evento de sincronización. Esta sincronía estricta simplifica enormemente el razonamiento formal sobre el sistema, ya que en cualquier punto de la ejecución se puede determinar con precisión qué comunicaciones ya ocurrieron y cuáles están pendientes.

El impacto práctico de CSP ha sido considerable a pesar de su origen puramente teórico: dio lugar al lenguaje Occam en los años 80, pensado específicamente para programar procesadores transputadores, y más recientemente inspiró directamente el diseño de los channels y las goroutines de Go, que son básicamente una implementación práctica y relajada (permiten tanto canales bloqueantes como con buffer) de las ideas originales de Hoare. Existen además herramientas de verificación formal construidas sobre esta teoría, como FDR (Failures-Divergence Refinement), que permiten comprobar computacionalmente si un sistema especificado en CSP cumple determinadas propiedades de seguridad y ausencia de bloqueos antes de implementarlo.

Memoria transaccional

La memoria transaccional traslada al ámbito de la memoria compartida una idea tomada directamente de las bases de datos: en lugar de proteger manualmente cada sección crítica con locks explícitos, el programador delimita un bloque de instrucciones como una transacción, y es el propio sistema, ya sea mediante software (STM, Software Transactional Memory) o con soporte directo del hardware del procesador quien garantiza que ese bloque completo se ejecute de forma atómica: o bien todas sus operaciones se aplican como si fueran una sola operación indivisible, o ninguna se aplica en absoluto. Internamente, el sistema suele llevar un registro de qué posiciones de memoria lee y escribe cada transacción mientras se ejecuta (de forma optimista, es decir, sin bloquear nada por adelantado), y solo al finalizar comprueba si hubo algún conflicto con otra transacción concurrente que haya tocado las mismas posiciones.

Si se detecta un conflicto, por ejemplo, dos transacciones que intentaron modificar la misma variable al mismo tiempo, el sistema aborta automáticamente una de ellas, descarta cualquier cambio parcial que hubiera hecho y la reintenta desde el principio, generalmente de forma completamente transparente para el programador. Esta estrategia optimista es la gran ventaja del modelo frente a los locks tradicionales: en la mayoría de los casos reales los conflictos son poco frecuentes, así que las transacciones suelen completarse sin reintentos, y se elimina de raíz buena parte de los problemas asociados al uso manual de candados, como los interbloqueos (no puede haber un interbloqueo si nunca se retiene un lock esperando a que otro proceso libere otro) o los errores sutiles de olvidar liberar un lock en alguna ruta de código.

La contrapartida es que el rendimiento del modelo depende fuertemente de dos factores: la tasa real de conflictos entre transacciones (si son muy frecuentes, el sistema pasa más tiempo reintentando que avanzando) y si existe o no soporte del procesador para acelerar estas operaciones a nivel de hardware los procesadores modernos han empezado a incorporar instrucciones específicas para este propósito, ya que una implementación puramente en software suele tener una

sobrecarga considerable comparada con el acceso a memoria ordinario. Por eso suele describirse como una técnica todavía en expansión más que como un mecanismo ya universalmente adoptado, aunque cada vez con mayor presencia tanto en investigación como en lenguajes y librerías concretas (Clojure y Haskell, por ejemplo, ofrecen soporte nativo o mediante librerías para memoria transaccional).

Ventajas de la ejecución de concurrencia

Mejor aprovechamiento de recursos y del tiempo

Un ejemplo numérico ilustra bien esta ventaja: si tres trabajos con distintos perfiles (uno con mucho cómputo, otro con mucha E/S por terminal, otro con mucha E/S por disco e impresora) se ejecutan uno tras otro, tardan 30 minutos en total y el uso del procesador ronda apenas el 20%, con memoria, disco e impresora usados solo un 33% del tiempo. Al ejecutarlos concurrentemente bajo multiprogramación, aprovechando que mientras un trabajo espera una operación de E/S el procesador puede atender a otro, el mismo trabajo total se completa en solo 15 minutos: el uso del procesador se duplica al 40%, el de memoria, disco e impresora sube al 67%, la productividad pasa de 6 a 12 trabajos por hora, y el tiempo de respuesta medio baja de 18 a 10 minutos.

Ejecutar tareas concurrentemente, en lugar de secuencialmente, permite exprimir muchísimo mejor un mismo hardware, especialmente porque el procesador nunca se queda ocioso esperando a un dispositivo de E/S: en cuanto un trabajo se bloquea por eso, el sistema operativo simplemente le cede el procesador a otro.

Rendimiento en la gestión de hilos

Comparado con crear procesos completos, gestionar concurrencia mediante hilos dentro de un mismo proceso trae varias ventajas de rendimiento muy concretas: crear un hilo nuevo es mucho más rápido que crear un proceso nuevo (los estudios del sistema Mach mostraron que un hilo se crea diez veces más rápido que un proceso en UNIX), finalizar un hilo también toma menos tiempo que finalizar

un proceso, y cambiar de contexto entre dos hilos del mismo proceso es más rápido que cambiar entre procesos distintos.

Además, como los hilos de un mismo proceso comparten memoria y archivos, pueden comunicarse entre sí sin necesidad de invocar al núcleo del sistema operativo, a diferencia de la comunicación entre procesos independientes, que sí requiere esa intervención para dar protección y mecanismos de comunicación, lo que hace mucho más eficiente construir una aplicación como un conjunto de hilos que como un conjunto de procesos separados. Un servidor de archivos es un caso representativo: cada nueva petición puede atenderse lanzando un hilo nuevo, y si la máquina es multiprocesador, varios hilos del mismo proceso pueden ejecutarse simultáneamente en distintos núcleos.

Razones para permitir la cooperación entre procesos

Existen varias razones concretas por las que conviene diseñar un entorno que permita que los procesos cooperen entre sí en lugar de ejecutarse de forma completamente aislada:

- Compartir información: cuando varios usuarios están interesados en el mismo dato (por ejemplo, un archivo compartido), es necesario permitir el acceso concurrente a esa información.
- Acelerar los cálculos: si se quiere que una tarea se complete más rápido, se puede dividir en subtareas que se ejecuten en paralelo, aunque esta aceleración solo es real si la computadora dispone de varios elementos de procesamiento (varias CPU o varios canales de E/S).
- Modularidad: permite construir el sistema dividiendo sus funciones en procesos o hilos separados, en vez de como un bloque monolítico.
- Conveniencia: incluso un único usuario puede querer trabajar en varias tareas a la vez por ejemplo, editar, imprimir y compilar al mismo tiempo.

Detección y explotación del paralelismo

Los multiprocesadores hacen posible explotar el paralelismo, y en la práctica los sistemas de cómputo obtienen los beneficios del procesamiento concurrente sobre todo mediante la multiprogramación de varios procesos, más que por explotar paralelismo dentro de un único proceso. Un diseño modular basado en esta idea aporta, además, flexibilidad y facilita la expansión de la capacidad del sistema a futuro.

Tipos de procesos concurrentes

Los procesos concurrentes se clasifican según el grado en que perciben la existencia de los demás y la forma en que interactúan entre sí. Existen tres tipos:

1. Procesos que no se perciben entre sí (competencia)

Son procesos independientes que no están diseñados para trabajar juntos: cada uno ejecuta su propia tarea sin saber que existen los otros. El ejemplo típico es la multiprogramación de varios trabajos independientes (por lotes, sesiones interactivas, o una mezcla de ambos). Aunque no cooperan, entran en competencia porque disputan el acceso a los mismos recursos físicos, un disco, un fichero, una impresora, y el sistema operativo debe regular ese acceso.

No hay intercambio de información entre ellos, pero la ejecución de uno sí puede afectar al comportamiento del otro: si dos procesos quieren el mismo recurso único, uno de ellos deberá esperar. Esta situación genera tres problemas de control:

- Exclusión mutua: solo un proceso a la vez puede usar el recurso crítico (por ejemplo, una impresora), evitando que sus operaciones se intercalen.
- Interbloqueo: puede ocurrir cuando dos procesos necesitan dos recursos cada uno, y cada uno retiene el recurso que el otro necesita mientras espera el que le falta, quedando ambos bloqueados indefinidamente.

- Inanición: un proceso puede quedar esperando el recurso indefinidamente si el sistema operativo sigue concediendo el acceso a otros procesos antes que a él, aunque nunca llegue a producirse un interbloqueo real.

2. Procesos que se perciben indirectamente entre sí (cooperación por compartición)

Son procesos que no necesariamente conocen la identidad de los demás, pero comparten acceso a algún objeto común (una variable, un fichero, una base de datos) y saben que otros procesos pueden acceder a ese mismo dato. Por eso deben cooperar para garantizar que los datos compartidos se manipulen correctamente.

Aquí se presentan de nuevo los problemas de exclusión mutua, interbloqueo e inanición, con la diferencia de que el dato puede accederse de dos formas (lectura y escritura), y solo las escrituras necesitan ser mutuamente exclusivas. Pero surge además un problema adicional propio de este tipo: la coherencia de datos. Por ejemplo, si dos variables a y b deben mantener siempre la relación a = b, y dos procesos actualizan ambas variables por separado, aunque cada uno respete la exclusión mutua sobre cada variable individual, el resultado final puede violar esa relación si sus operaciones se intercalan de cierta forma. La solución consiste en tratar la secuencia completa de operaciones relacionadas como una única sección crítica.

3. Procesos que se perciben directamente entre sí (cooperación por comunicación)

Son procesos capaces de identificarse mutuamente (por ejemplo, mediante su ID de proceso) y que están diseñados explícitamente para trabajar juntos en una actividad común, coordinándose mediante el intercambio de mensajes, típicamente a través de primitivas de envío y recepción proporcionadas por el lenguaje de programación o por el núcleo del sistema operativo.

A diferencia de los dos casos anteriores, aquí los procesos no comparten memoria ni recursos directamente, por lo que la exclusión mutua deja de ser un requisito de control. Sin embargo, persisten los otros dos problemas:

- Interbloqueo: puede darse si dos procesos quedan bloqueados esperando cada uno una comunicación del otro.
- Inanición: por ejemplo, si tres procesos intentan comunicarse entre sí, pero dos de ellos intercambian información repetidamente mientras el tercero queda indefinidamente a la espera de que alguno le responda.

Tipos de interacciones entre procesos

Aquí se las interacciones se diferencian de la misma manera que los procesos, resumido brevemente podemos notar sus diferencias fácilmente:

Competencia.	Cooperación por compartición.	Cooperación por comunicación.
Los procesos no se perciben entre sí.	Los procesos se perciben indirectamente a través de un objeto común.	Los procesos se perciben directamente entre sí.
No hay intercambio de información.	Los procesos deben coordinarse para que los datos se mantengan correctos.	Se coordinan mediante el intercambio de mensajes.
Si dos procesos quieren el mismo recurso, uno debe esperar.	Solo las escrituras requieren exclusión mutua estricta.	Solo intercambian mensaje, por lo tanto, la exclusión mutua no es necesaria.
Exige garantizar la exclusión mutua, evitar interbloqueos y evitar inanición.	Se debe tratar toda la secuencia de operaciones relacionadas como una única sección crítica.	Se necesita evitar el interbloqueo y la inanición.

Tabla 2. Tipos de interacciones entre procesos y sus características.

Cuadro Sinóptico

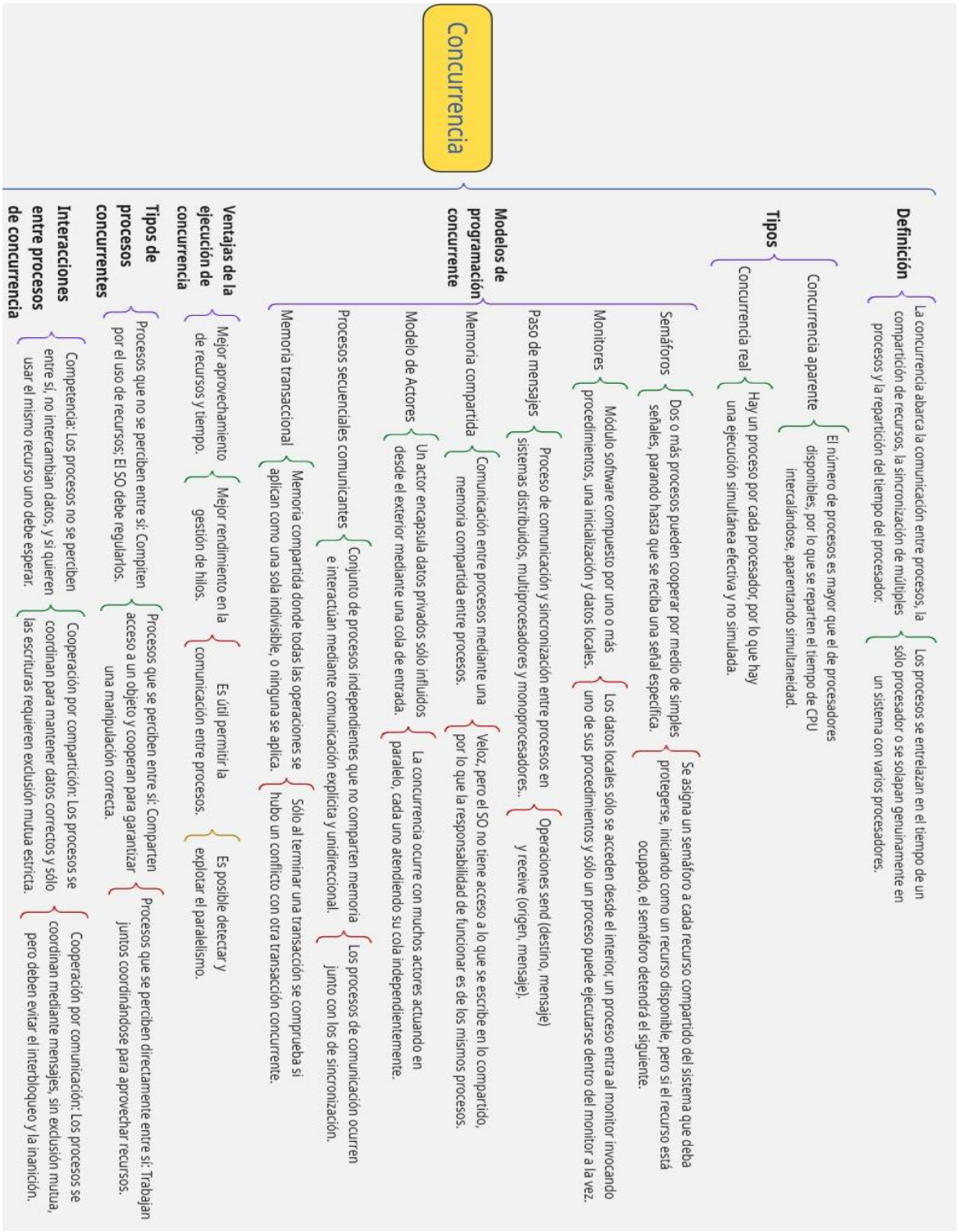


Imagen 14. Cuadro Sinóptico acerca de la concurrencia en Sistemas Operativos.

Conclusión General

Después de revisar la forma en que los sistemas operativos administran sus procesos, queda claro que detrás de algo que parece tan simple (como abrir varios programas y que todos funcionen "al mismo tiempo"), hay una cantidad enorme de mecanismos trabajando en conjunto. Desde el Bloque de Control de Proceso, que guarda toda la información necesaria para que ningún proceso pierda su lugar, hasta los distintos modos de ejecución que protegen al núcleo del sistema de los programas de usuario, se puede ver que el sistema operativo tiene que ser extremadamente cuidadoso y ordenado para que todo funcione de manera correcta. Se puede notar que no existe una única forma de resolver estos problemas. Arquitecturas como los micronúcleos muestran que es posible construir un sistema operativo más ligero y modular, delegando funciones a servidores en espacio de usuario, mientras que sistemas como Linux o Solaris han desarrollado sus propias soluciones para el manejo de hilos y para el multiprocesamiento simétrico, adaptándose a las necesidades del hardware moderno con varios procesadores.

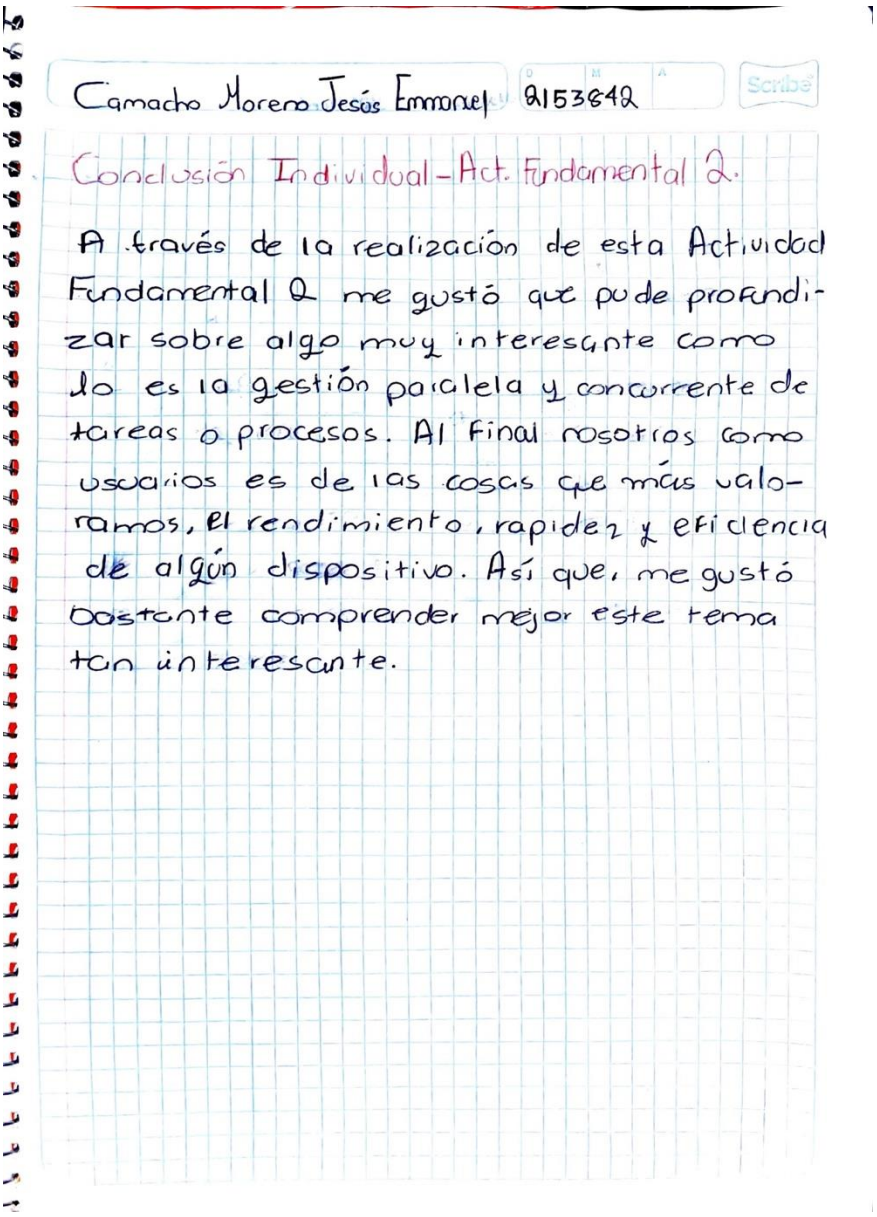
En cuanto a la concurrencia, es un tema que va mucho más allá de simplemente "ejecutar cosas al mismo tiempo". Involucra decisiones de diseño importantes sobre cómo los procesos van a comunicarse, cómo van a compartir recursos y cómo se va a evitar que se bloqueen entre sí de forma indefinida. Los distintos modelos de programación concurrente que existen (semáforos, monitores, paso de mensajes, memoria compartida, el modelo de actores, procesos secuenciales comunicantes y memoria transaccional) demuestran que a lo largo de los años se han buscado diferentes caminos para resolver básicamente el mismo problema, cada uno con ventajas y desventajas dependiendo del contexto en el que se use.

Finalmente, se puede afirmar que ejecutar procesos de forma concurrente no es solo una opción, sino una necesidad para aprovechar mejor los recursos de

cualquier sistema, reducir tiempos de espera y permitir que las aplicaciones sean más rápidas y eficientes.

Conclusiones Individuales

Camacho Moreno Jesús Emmanuel



Camacho Moreno Jesús Emmanuel 2153642

Conclusión Individual - Act. Fundamental 2.

A través de la realización de esta Actividad Fundamental 2 me gustó que pude profundizar sobre algo muy interesante como lo es la gestión paralela y concurrente de tareas o procesos. Al final nosotros como usuarios es de las cosas que más valoramos, el rendimiento, rapidez y eficiencia de algún dispositivo. Así que, me gustó bastante comprender mejor este tema tan interesante.

Cardenas Vazquez Carlos Tomas 2226997

Conclusión AP2

El desarrollo de esta actividad permitio comprender la capacidad de respuesta de los Sistemas Operativos es gracias a la rigurosa arquitectura interna. La multitarea pues no ocurre por casualidad sino que el sistema opera como un coordinador central que administra los ciclos de CPU, ha memoria y el hardware, logrando que múltiples aplicaciones se ejecuten de manera simultanea. sin interferencias ni fallos.

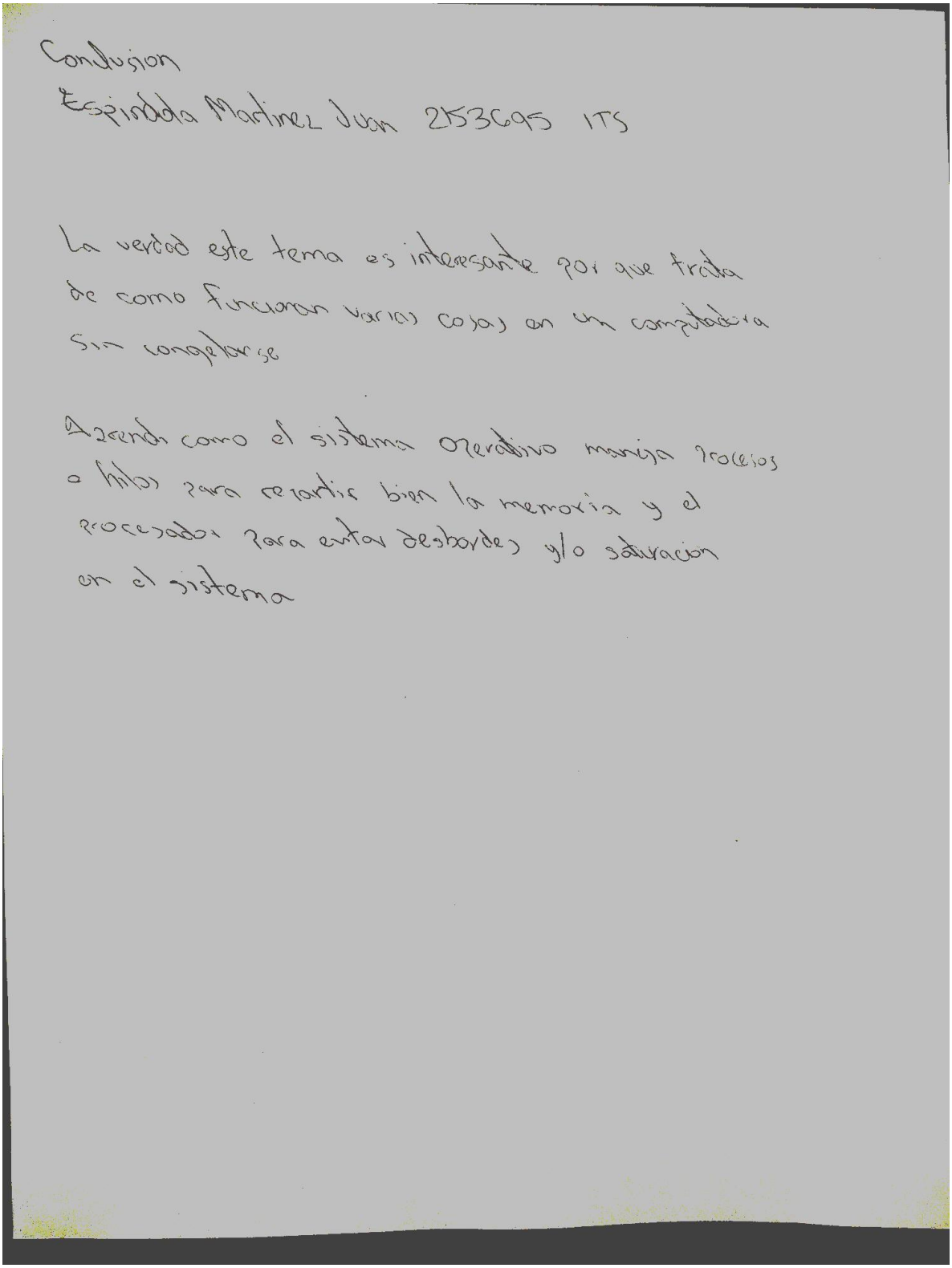
En conclusion, dominar la administración de hilos y la sincronización de recursos es algo vital. Estos conceptos constituyen la base para desarrollar e implementar soluciones informáticas eficientes y fiables, capaces de responder a las elevadas exigencias de la tecnologia actual.

Juan Kristof Cerda Herrera 2050864

Conclusión AF2

A lo largo de este tema fui entendiendo que el funcionamiento de una computadora es mucho más complejo de lo que parece cuando simplemente abrimos un programa o usamos varias aplicaciones al mismo tiempo.
Al principio conceptos como procesos, hilos o concurrencia pueden parecer solamente terminos tecnicos, pero al relacionarlos con situaciones que pasan todos los dias en una computadora, es más facil entender para que sirven realmente.

Otro aspecto que considero importante es el multiprocesamiento simetrico, ya que permite volver aprovechar varios procesadores para realizar diferentes tareas.
Junto con los micronucleos, estos temas me hicieron ver que existen diferentes formas de ver y diseñar un sistema operativo.



Conclusion
Espindola Martinez Juan 2553695 ITS

La verdad este tema es interesante por que trata de como funcionan varias cosas en un computadora sin congelarse

Aprendi como el sistema operativo maneja procesos o hilos para repartir bien la memoria y el procesador para evitar desbordes y/o saturacion en el sistema

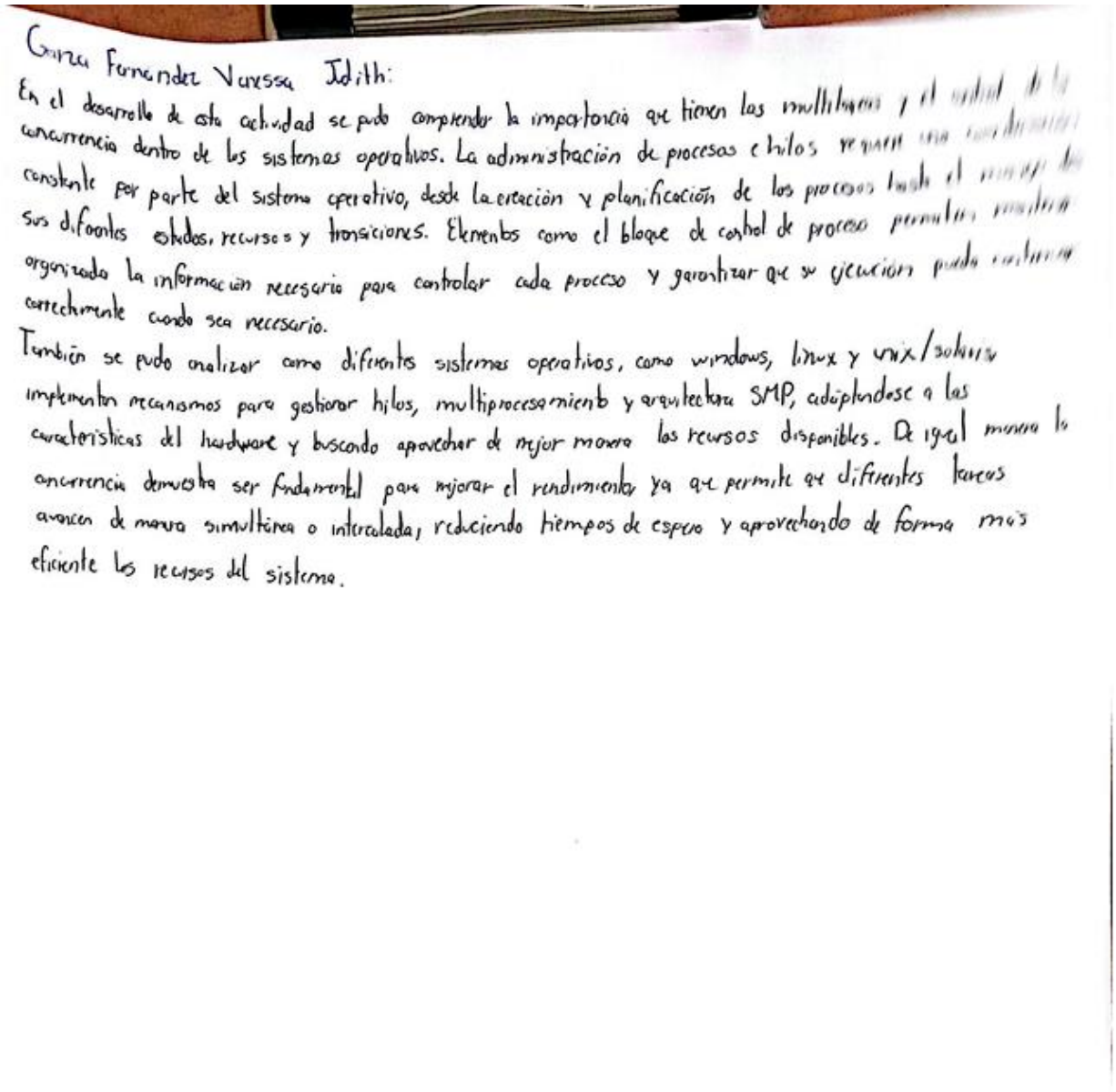
Garza Arizpe Emilio - 2143702

Conclusión AF2

En esta actividad lo que me pareció más interesante fue comprender cómo el sistema operativo logra coordinar tantas tareas al mismo tiempo sin que todo colapse. Analizando conceptos como los procesos e hilos, te das cuenta que toda la lógica y la eficiencia que hay detrás para aprovechar cada ciclo del procesador.

También algo importante de la actividad, fue el control de concurrencia. Entender mecanismos como los semáforos, los monitores o el paso de mensajes, te deja más claro que hay detrás para evitar problemas como el deadlock.

Comparar arquitecturas entre sistemas operativos también considero que fue importante ya que te hace ver que no hay una solución "correcta" sino distintos enfoques adaptados a diferentes necesidades. Entendí mejor como funciona internamente un equipo de computo.



Garza Fernandez Vanessa Judith:

En el desarrollo de esta actividad se pudo comprender la importancia que tienen los multihilos y el control de la concurrencia dentro de los sistemas operativos. La administración de procesos e hilos requiere una coordinación constante por parte del sistema operativo, desde la creación y planificación de los procesos hasta el manejo de sus diferentes estados, recursos y transiciones. Elementos como el bloque de control de proceso permiten mantener organizada la información necesaria para controlar cada proceso y garantizar que su ejecución pueda continuar correctamente cuando sea necesario.

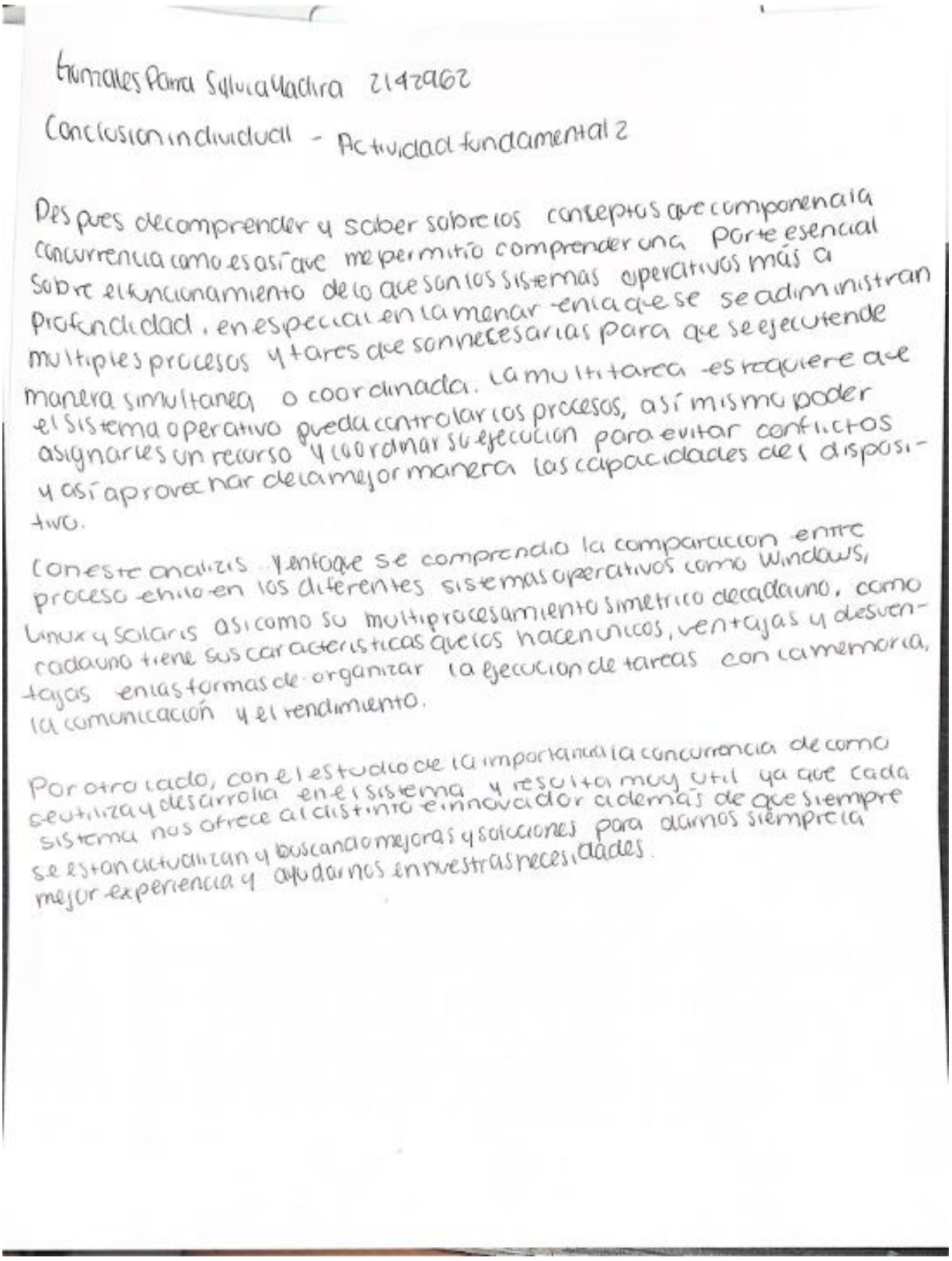
También se pudo analizar como diferentes sistemas operativos, como windows, linux y unix/solaris implementan mecanismos para gestionar hilos, multiprocesamiento y arquitectura SMP, adaptándose a las características del hardware y buscando aprovechar de mejor manera los recursos disponibles. De igual manera la concurrencia demuestra ser fundamental para mejorar el rendimiento ya que permite que diferentes tareas avancen de manera simultánea o intercalada, reduciendo tiempos de espera y aprovechando de forma más eficiente los recursos del sistema.

Gonzalez Martinez Mario Ivan - 2117170

Conclusion Individual - Actividad Fundamental 2

Durante la realizacion de esta actividad pude y tuve la oportunidad de comprender que el rendimiento y la estabilidad de un sistema operativo dependen directamente de su capacidad para gestionar la multitarea y la concurrencia. Analizar el papel del Bloque de Control de Proceso, los mecanismos de cambio de contexto y las diferencias de diseño en plataformas como Windows, Linux, Solaris evidencia la complejidad detrás de la asignación eficiente de CPU y memoria. Asimismo, explorar las distintas herramientas de sincronización y modelos de programación concurrente deja claro que prevenir problemas como los interbloqueos e inanición es fundamental. Dominar estos conceptos es clave para diseñar e implementar software optimizado y capaz de aprovechar la infraestructura del multiprocesador moderno

González Parra Sylvia Yadira



Tornales Para Sylvia Yadira 2142962

Conclusion individual - Actividad fundamental 2

Despues de comprender y saber sobre los conceptos que componen a la concurrencia como es así que me permitió comprender una parte esencial sobre el funcionamiento de lo que son los sistemas operativos más a profundidad, en especial en la menar-enia que se administran multiples procesos y tares que son necesarias para que se ejecute de manera simultanea o coordinada. La multitarea -es requiere que el sistema operativo pueda controlar los procesos, así mismo poder asignarles un recurso y coordinar su ejecución para evitar conflictos y así aprovechar de la mejor manera las capacidades del dispositivo.

Con este analizis y enfoque se comprendio la comparación entre proceso e hilo en los diferentes sistemas operativos como Windows, Linux y Solaris así como su multiprocesamiento simetrico de cada uno, como cada uno tiene sus caracteristicas que los hacen unicos, ventajas y desventajas en las formas de organizar la ejecución de tareas con la memoria, la comunicación y el rendimiento.

Por otro lado, con el estudio de la importancia la concurrencia de como se utiliza y desarrolla en el sistema y resulta muy util ya que cada sistema nos ofrece al distinto e innovador ademas de que siempre se estan actualizan y buscando mejoras y soluciones para darnos siempre la mejor experiencia y ayudarnos en nuestras necesidades.

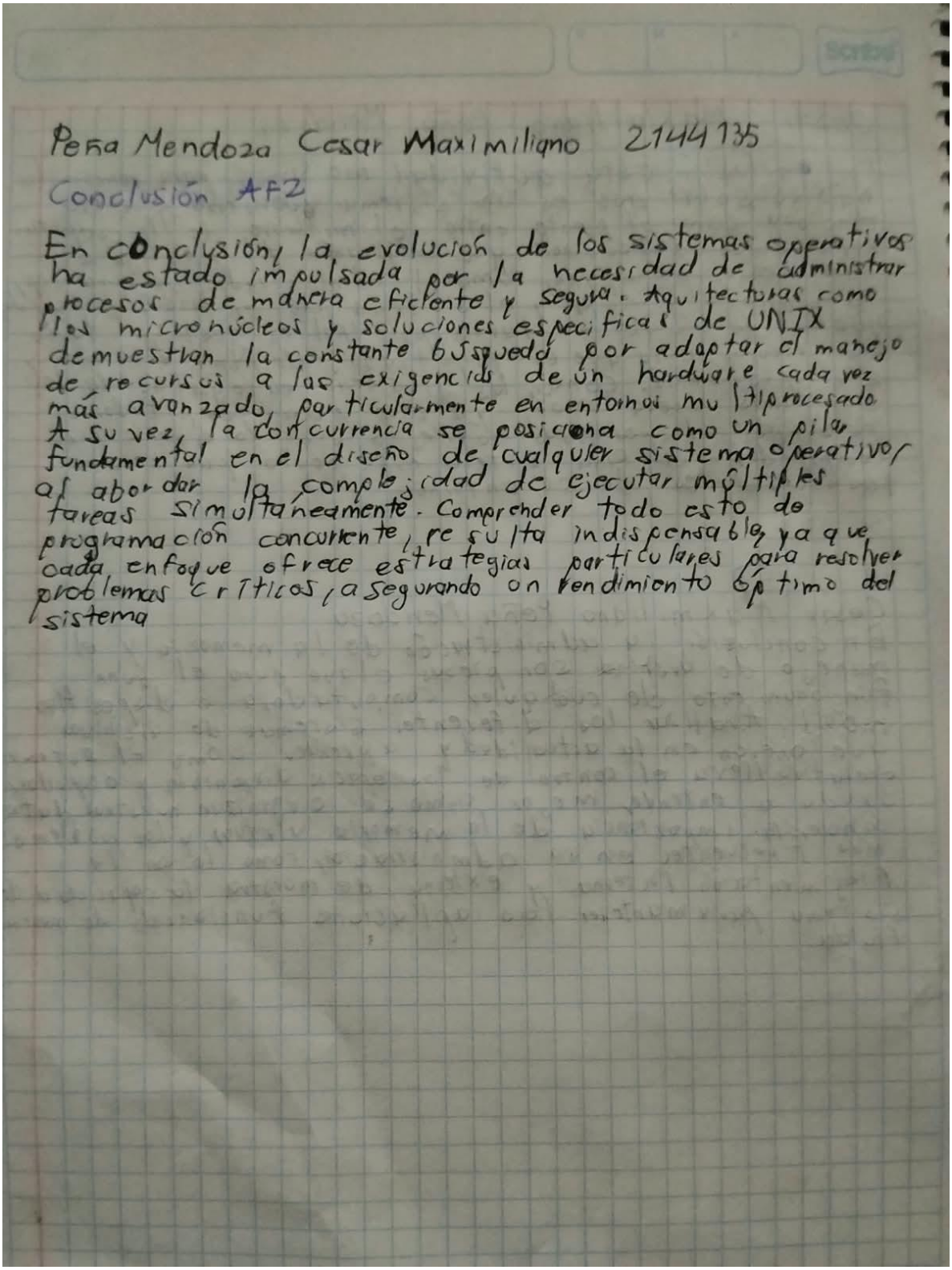
López Guzmán Eder Abihú - 2116464

Conclusión.

Con esta actividad comprendí a mayor profundidad el concepto de concurrencia, entendiendo que es todo un problema de diseño que el sistema operativo tiene que resolver cuidadosamente.

Uno de los conceptos que me parecieron más interesantes fue el modelo de programación concurrente, porque no hay una única forma "correcta" de resolver la sincronización entre procesos, sino que cada modelo tiene su propio enfoque y sus propias ventajas dependiendo de la situación.

En conclusión, esta actividad plantea correctamente los conceptos clave que destaquen el programar pensando en cómo los procesos van a compartir recursos entre sí.

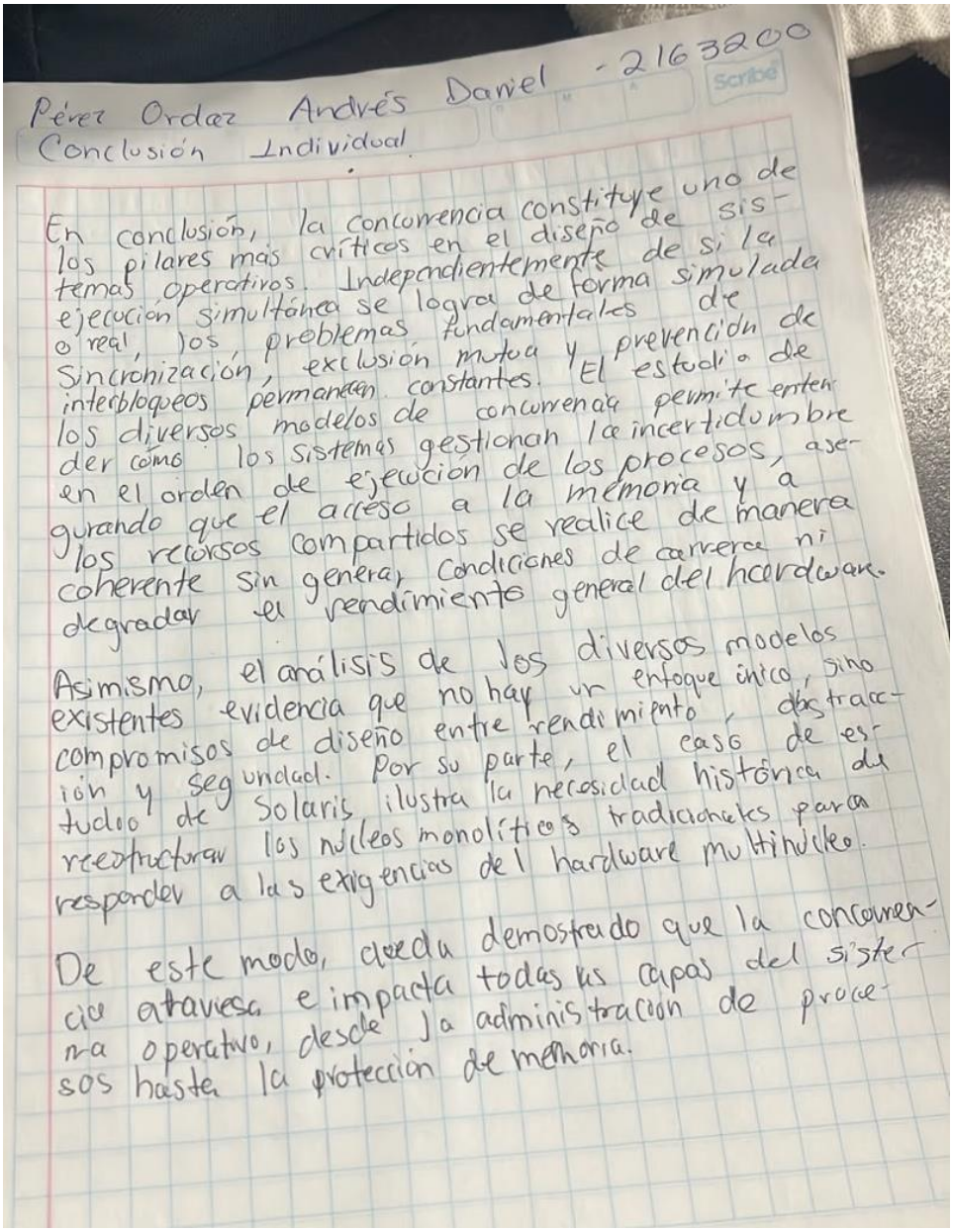


Peña Mendoza Cesar Maximiliano 2144135

Conclusión AF2

En conclusión, la evolución de los sistemas operativos ha estado impulsada por la necesidad de administrar procesos de manera eficiente y segura. Arquitecturas como los microkúcleos y soluciones específicas de UNIX demuestran la constante búsqueda por adaptar el manejo de recursos a las exigencias de un hardware cada vez más avanzado, particularmente en entornos multiprocesado A su vez, la concurrencia se posiciona como un pila fundamental en el diseño de cualquier sistema operativo, al abordar la complejidad de ejecutar múltiples tareas simultáneamente. Comprender todo esto de programación concurrente, resulta indispensable, ya que cada enfoque ofrece estrategias particulares para resolver problemas críticos, asegurando un rendimiento óptimo del sistema

Pérez Ordaz Andrés Daniel



Pérez Ordaz Andrés Daniel - 2163200

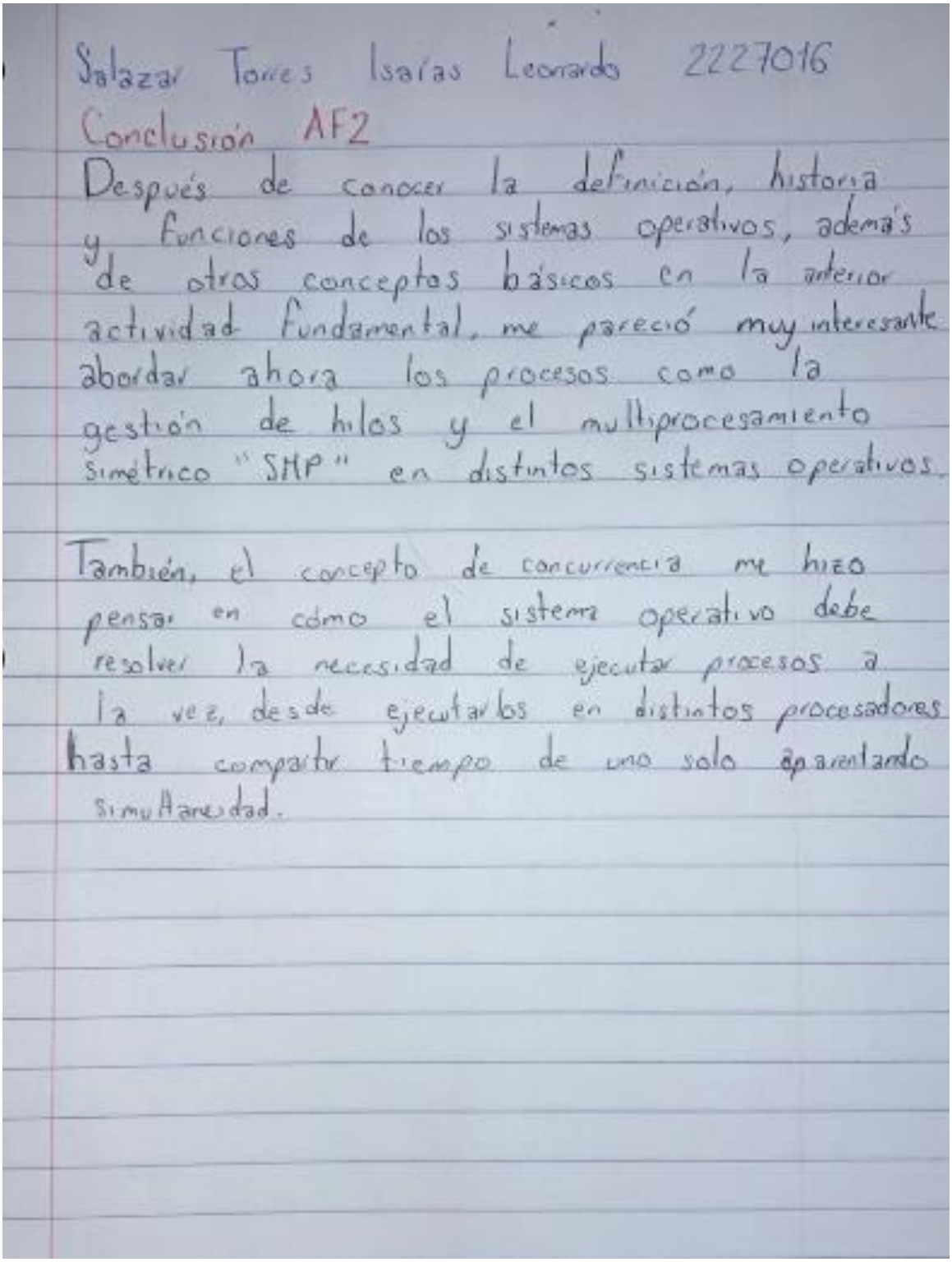
Conclusión Individual

En conclusión, la concurrencia constituye uno de los pilares más críticos en el diseño de sistemas operativos. Independientemente de si la ejecución simultánea se logra de forma simulada o real, los problemas fundamentales de sincronización, exclusión mutua y prevención de interbloqueos permanecen constantes. El estudio de los diversos modelos de concurrencia permite entender cómo los sistemas gestionan la incertidumbre en el orden de ejecución de los procesos, asegurando que el acceso a la memoria y a los recursos compartidos se realice de manera coherente sin generar condiciones de carrera ni degradar el rendimiento general del hardware.

Asimismo, el análisis de los diversos modelos existentes evidencia que no hay un enfoque único, sino compromisos de diseño entre rendimiento, abstracción y seguridad. Por su parte, el caso de estudio de Solaris ilustra la necesidad histórica de reestructurar los núcleos monolíticos tradicionales para responder a las exigencias del hardware multinúcleo.

De este modo, queda demostrado que la concurrencia atraviesa e impacta todas las capas del sistema operativo, desde la administración de procesos hasta la protección de memoria.

Salazar Torres Isaias Leonardo


Salazar Torres Isaias Leonardo 2227016

Conclusión AF2

Después de conocer la definición, historia y funciones de los sistemas operativos, además de otros conceptos básicos en la anterior actividad fundamental, me pareció muy interesante abordar ahora los procesos como la gestión de hilos y el multiprocesamiento simétrico "SMP" en distintos sistemas operativos.

También, el concepto de concurrencia me hizo pensar en cómo el sistema operativo debe resolver la necesidad de ejecutar procesos a la vez, desde ejecutarlos en distintos procesadores hasta compartir tiempo de uno solo aparentando simultaneidad.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill Interamericana.

Stallings, W. (2005). *Sistemas operativos: Aspectos internos y principios de diseño* (5.ª ed.). Pearson Educación.

Tanenbaum, A. S. (2009). Sistemas operativos modernos (3.ª ed.). Pearson Educación de México.

Carretero Pérez, J., García Carballeira, F., de Miguel Anasagasti, P., y Pérez Costoya, F. (2001). Sistemas operativos: Una visión aplicada. McGraw-Hill/Interamericana de España.

La Red Martínez, D. L. (2004). Sistemas operativos. Universidad Nacional del Nordeste, Facultad de Ciencias Exactas y Naturales y Agrimensura, Departamento de Informática.